

# Estructura de un programa informático. Tipo de datos simples

Joan Arnedo Moreno

Programación básica (ASX)  
Programación (DAM)  
Programación (DAW)



## Índice

|                                                     |    |
|-----------------------------------------------------|----|
| Introducción                                        | 5  |
| Resultados de aprendizaje                           | 7  |
| 1 Iniciación a la programación                      | 9  |
| 1.1 ¿Qué es un programa?                            | 9  |
| 1.2 Tipos de comandos que acepta un ordenador       | 10 |
| 1.3 Crear un programa ejecutable...                 | 12 |
| 1.3.1 ...en lenguaje de máquina                     | 13 |
| 1.3.2 ... mediante un lenguaje compilado            | 15 |
| 1.3.3 ... mediante un lenguaje interpretado         | 18 |
| 1.3.4 Entornos integrados de edición                | 19 |
| 1.4 Su primer programa                              | 20 |
| 1.4.1 Características relevantes del Java           | 21 |
| 1.4.2 Creación y ejecución de programas Java        | 22 |
| 1.4.3 ¡Hola, mundo!                                 | 24 |
| 2 Manipulación básica de datos                      | 29 |
| 2.1 Tipos de datos                                  | 29 |
| 2.1.1 El tipo de dato booleano                      | 31 |
| 2.1.2 El tipo de dato entero                        | 31 |
| 2.1.3 El tipo de dato real                          | 32 |
| 2.1.4 El tipo de dato carácter                      | 33 |
| 2.2 Transformación de los datos                     | 34 |
| 2.2.1 Operaciones entre booleanos                   | 35 |
| 2.2.2 Operaciones entre enteros                     | 36 |
| 2.2.3 Operaciones entre reales                      | 38 |
| 2.2.4 Operaciones entre caracteres                  | 38 |
| 2.2.5 Construcción de expresiones                   | 39 |
| 2.2.6 Evaluación de expresiones                     | 40 |
| 2.2.7 Desbordamientos y errores de precisión        | 42 |
| 2.3 Gestión de datos en la memoria                  | 43 |
| 2.3.1 Cómo funciona la memoria del ordenador        | 44 |
| 2.3.2 Declaración de variables                      | 45 |
| 2.3.3 Identificadores                               | 47 |
| 2.3.4 Ús de variables                               | 48 |
| 2.3.5 Constantes                                    | 51 |
| 2.4 Conversiones de tipo                            | 54 |
| 2.4.1 Conversión implícita                          | 54 |
| 2.4.2 Conversión explícita                          | 55 |
| 2.4.3 Conversión con tipos no numéricos             | 57 |
| Visualización y entrada de los datos en Java        | 58 |
| 2.5.1 Instrucciones de salida de datos por pantalla | 58 |

|                                                          |    |
|----------------------------------------------------------|----|
| 2.5.2 Tipo de dato avanzados: cadenas de texto . . . . . | 60 |
| 2.5.3 Entrada simple de datos por teclado . . . . .      | 63 |
| 2.6 Soluciones de los retos propuestos . . . . .         | 65 |

## Introducción

Esta unidad formativa representa el punto de partida para entender cómo puede desarrollar un programa de ordenador que permita llevar a cabo una tarea de cierta complejidad. Suele decirse que programar tiene una parte de arte, ya que este proceso requiere un grado no despreciable de creatividad. Lo cierto es que, como en otras muchas disciplinas, incluidas las artísticas, ser un buen programador implica practicar y practicar muchas horas. Sólo leyendo un manual no es suficiente ni mucho menos. Ahora bien, a pesar de la necesidad de este grado de creatividad, sí que hay una serie de metodologías y fundamentos teóricos básicos que deben tener claros para poder plantearse cómo empezar. Estos fundamentos serán de gran utilidad posteriormente para facilitar el trabajo de entender y mejorar la técnica adquirida.

Para aprender a programar primero es necesario empezar por herramientas más sencillas disponibles para crear un programa, y poco a poco ir asimilando más complejas. Es importante que esta progresión se haga de forma escalonada para garantizar que se han entendido los conceptos. Por tanto, en esta unidad usará el ordenador prácticamente como si fuera una calculadora. Sin embargo, esto será suficiente para que veáis algunos de los aspectos básicos que le acompañarán siempre, como es saber gestionar datos básicos mediante un programa.

El núcleo formativo “Iniciación a la programación” proporciona una visión partiendo desde cero de qué es un programa de ordenador, cómo funciona y cuál es el proceso de creación a grandes rasgos. Como hilo conductor para ver todos estos conceptos se usan las diferentes formas en que se pueden clasificar los mecanismos para crear un programa. Para llevar a la práctica todo esto, ya antes de terminar el núcleo se sentará delante del ordenador y creará un primer programa muy sencillo.

Dentro del núcleo “Manipulación básica de datos” ya empezará a ver cómo se estructuran los datos dentro del ordenador y cómo es posible transformarlos para realizar ciertos cálculos sencillos y mostrar el resultado por la pantalla. Dado que, en última instancia, todo lo que hace un ordenador es manipular datos, este núcleo es especialmente importante de cara a disponer de los conceptos fundamentales para poder entender partes más complejas de la programación.

Teniendo en cuenta la necesidad de practicar delante de un ordenador para entender los diferentes conceptos expuestos en esta unidad, a lo largo del texto encontrará una serie de retos en los que le planteamos pequeños ejercicios para complementar cada explicación concreta. Es importante que dedique un rato a resolverlos antes de seguir adelante. En cualquier caso, también es muy recomendable que intente hacer los ejemplos que irá encontrando en su entorno de trabajo.



## Resultados de aprendizaje

Al finalizar esta unidad, el alumno/a:

1. Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.
2. Identifica los bloques que componen la estructura de un programa informático.
3. Crea proyectos de desarrollo de aplicaciones y utiliza entornos integrados de desarrollo.
4. Identifica los distintos tipos de variables y la utilidad específica de cada uno.
5. Modifica el código de un programa para crear y utilizar variables.
6. Crea y utiliza constantes y literales.
7. Clasifica, reconoce y utiliza en expresiones a los operadores del lenguaje.
8. Comprueba el funcionamiento de las conversiones de tipo explícito e implícito.
9. Introduce comentarios en el código.



## 1. Iniciación a la programación

Como en cualquier proceso de aprendizaje, es necesario empezar por el principio. Es importante tener claros un conjunto de conceptos básicos que ayuden a comprender los conceptos más avanzados que vendrán posteriormente. En este caso, se trata de establecer qué es un programa, cómo funciona y cuál es el modelo general para crear uno. Sólo una vez lo tenga claro se puede plantear sentarse delante del ordenador y empezar a programar.

### 1.1 ¿Qué es un programa?

Un primer paso para empezar a estudiar cómo hacer un programa informático es tener claro qué es un programa. En contraste con otros términos usados en informática, es posible referirse a un “programa” en el lenguaje coloquial sin tener que estar hablando necesariamente de ordenadores. Podría estar refiriéndose al programa de un ciclo de conferencias o de cine. Pero, a pesar de no tratarse de un contexto informático, este uso ya le aporta una idea general de su significado: un conjunto de eventos ordenados de forma que suceden de forma secuencial en el tiempo, uno tras otro.

Otro uso habitual, ahora sí vinculado al contexto de las máquinas y los autómatas, podría ser para referirse al programa de una lavadora o de un robot de cocina. Pero en este caso lo que sucede es un conjunto no tanto de eventos, sino de órdenes que el electrodoméstico sigue ordenadamente. Una vez seleccionado el programa que queremos, el electrodoméstico realiza todas las tareas correspondientes de forma autónoma.

Por ejemplo, el programa de un robot de cocina para hacer una crema de maíz  
él sería:

1. Espera a que introduzca maíz y mantequilla.
2. Gira durante un minuto, avanzando progresivamente de la velocidad 1 a la 5.
3. Espera a que introduzca leche y sal.
4. Gira durante 30 segundos a velocidad 7.
5. Gira durante 10 minutos a velocidad 3 mientras cuece a una temperatura de 90 grados.
6. Se detiene. ¡La crema está lista!

Este conjunto de órdenes no es arbitrario, sino que sirve para llevar a cabo una tarea de cierta complejidad que no puede realizarse de una sola vez. Se debe dar paso por paso. Todas

las órdenes están vinculadas entre sí para llegar a alcanzar este objetivo y, sobre todo, es muy importante la disposición en la que se llevan a cabo.

Entrando ya, ahora sí, en el mundo de los ordenadores, la forma en que se estructura el tipo de tareas que éstos pueden realizar tiene mucho en común con los programas de electrodomésticos. Sin embargo, en este caso en lugar de transformar ingredientes (o lavar ropa sucia, si se tratara de una lavadora), lo que el ordenador transforma es información o datos. Un programa informático no es más que una serie de órdenes que se llevan a cabo secuencialmente, aplicadas sobre un conjunto de datos.

¿Qué datos procesa un programa informático? Bien, esto dependerá del tipo de programa:

- Un editor procesa datos de un documento de texto.
- Una hoja de cálculo procesa datos numéricos.
- Un videojuego procesa los datos que dicen la forma y ubicación de enemigos y jugadores, etc.
- Un navegador web procesa las órdenes del usuario y los datos que recibe desde de un servidor en Internet.

Por tanto, la tarea de un programador informático es escoger qué órdenes constituirán un programa de ordenador, en qué orden deben llevarse a cabo y sobre qué datos deben aplicarse, para que el programa lleve a cabo la tarea que debe resolver.

La dificultad de todo ello será mayor o pequeña dependiendo de la propia complejidad de lo que el programa debe hacer. No es lo mismo establecer qué debe hacer el ordenador para resolver una multiplicación de tres números que para procesar textos o visualizar páginas en Internet.

Por otra parte, una vez realizado el programa, cada vez que lo ejecute, el ordenador cumplirá todas las órdenes del programa.

Ejecutar un programa  
Para "ejecutar un programa"  
se entiende que el  
ordenador siga todas sus  
órdenes, desde la primera hasta la última

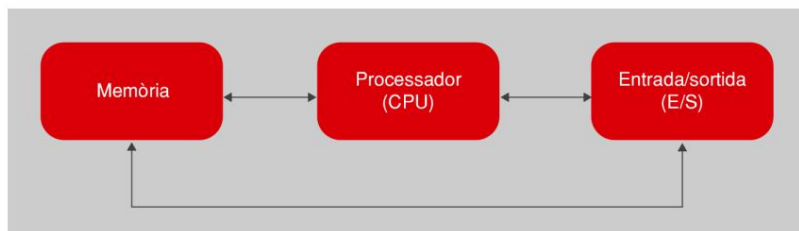
De hecho, un ordenador es incapaz de hacer absolutamente nada por sí mismo, siempre es necesario decirle qué debe hacer. Y esto se le dice mediante la ejecución de programas. Aunque desde el punto de vista del usuario puede parecer que cuando se pone en marcha un ordenador éste funciona sin ejecutar ningún programa concreto, debe tenerse en cuenta que su sistema operativo es un programa que está siempre en ejecución.

## 1.2 Tipos de órdenes que acepta un ordenador

Para realizar la tarea encomendada, un equipo puede aceptar diferentes tipos de mandatos. Éstas se encuentran limitadas a las capacidades de los componentes que lo conforman, al igual que el programa de una lavadora no puede incluir la orden de gratinar, ya que no tiene gratinador. Por tanto, es importante tener presente este hecho para saber qué se puede pedir al ordenador cuando cree un programa.

La estructura interna del ordenador se divide en una serie de componentes, todos comunicados entre sí, tal y como muestra la figura 1.1 de forma muy simplista, pero suficiente para empezar. Cada orden de un programa está vinculado de una u otra forma a alguno de estos componentes.

Figura 1. 1. Componentes básicos de un ordenador



El procesador es el centro neurálgico del ordenador y el elemento que es capaz de realizar las órdenes de manipulación y transformación de los datos. Un conjunto de datos se puede transformar de muchas formas, según las capacidades que ofrezca cada procesador. Sin embargo, hay muchas transformaciones que todos pueden hacer. Un ejemplo es la realización de operaciones aritméticas (suma, resta, multiplicación, división), tal y como realizan las calculadoras.

#### Procesador

El procesador también es conocido popularmente por sus siglas en inglés: CPU (central processing unit, unidad central de procesamiento).

La memoria permite almacenar datos mientras éstos no están siendo directamente manipulados por el procesador. Cualquier dato que debe ser tratado por un programa estará en la memoria. Mediante el programa se puede ordenar al procesador que guarde ciertos datos o que los recupere en cualquier momento.

Normalmente, cuando se habla de memoria a ese nivel nos referimos a memoria dinámica o RAM (random access memory, memoria de acceso aleatorio). Esta memoria no es persistente y una vez termina la ejecución del programa todos los datos con los que trataba se desvanecen. Por tanto, la información no se guardará entre sucesivas ejecuciones distintas de un mismo programa.

En ciertos contextos es posible que nos encontremos también con memoria ROM (read-only memory, memoria de sólo lectura). En ella, los datos están predefinidos de fábrica y no se puede almacenar nada, sólo podemos leer lo que contiene. Hay que decir que no es lo más habitual.

El sistema de entrada/salida (abreviado como E/S) permite el intercambio de datos con el exterior del ordenador, más allá del procesador y la memoria. Esto permite traducir la información procesada en acciones de control sobre cualquier periférico conectado al ordenador. Un ejemplo típico es establecer una vía de diálogo con el usuario, ya sea por medio del teclado o del ratón para pedirle información, como por la pantalla, para mostrar los resultados del programa. Este sistema es clave para convertir un ordenador en una herramienta de propósito general, ya que le capacita para controlar todo tipo de aparatos diseñados para conectarse a ellos.

Otra posibilidad importante del ordenador, dadas las limitaciones del sistema de memoria, es poder interactuar con el hardware de almacenamiento persistente de datos, como un disco duro.

Un ordenador es como una pizzería

Si se quiere hacer un símil con nuestro mundo de todos los días, un ordenador es como la cocina de una pizzería que acepta pedidos telefónicos. Realizar un pedido equivale a pedir el inicio de la ejecución de un programa. Para llevar a cabo este pedido, será necesario manipular una serie de ingredientes, que representarían los datos. El cocinero con sus utensilios (horno, amasador, etc.) serían el procesador, ya que manipulan y transforman los ingredientes. La nevera, los armarios o los contenedores, de donde el cocinero puede sacar ingredientes o donde puede guardarlos mientras no los está manipulando, representarían la memoria. El sistema de entrada/salida serían los elementos de comunicación con el exterior de la pizzería, como el motorista que lleva la pizza o el teléfono que el cocinero puede utilizar para pedir que le lleven nuevos ingredientes cuando se le acaban, pedir información adicional al usuario ("Se acabó el pimiento, ¿va bien si ponemos cebolla?"), o avisar algo más de lo previsto). De hecho, continuando con el símil, el cocinero prepara una pizza siguiendo un conjunto de pasos. En este caso la receta son las órdenes que debe seguir el programa. Y si el cocinero no tiene la receta no puede realizar el pedido.

Lenguaje natural El lenguaje natural es aquel que utilizamos los humanos para comunicarnos habitualmente.

Partiendo de esta descripción de las tareas que puede realizar un ordenador según los elementos que lo componen, un ejemplo de programa para multiplicar dos números es el mostrado en la tabla 1.1. Lo tiene expresado en lenguaje natural. Nótese cómo los datos deben estar siempre almacenados en la memoria para poder operar en ellos.

Tabla 1. 1. Un programa que multiplica dos números usando lenguaje natural

| Orden para dar                                                     | Element que l'efectua |
|--------------------------------------------------------------------|-----------------------|
| 1. Lee un número del teclado.                                      | E/S (teclado)         |
| 2. Desa el nombre a la memòria.                                    | Memoria               |
| 3. Lee otro número del teclado.                                    | E/S (teclado)         |
| 4. Desa el nombre a la memòria.                                    | Memoria               |
| 5. Recupera los números de la memoria y realiza la multiplicación. | Procesador            |
| 6. Guarda el resultado en la memoria.                              | Memoria               |
| 7. Mostra el resultat a la pantalla.                               | E/S (pantalla)        |

1.3 Crear un programa ejecutable...

Para crear un programa es necesario establecer qué órdenes deben darse al ordenador y en qué secuencia. Ahora bien, hoy en día los ordenadores todavía no entienden el lenguaje natural (como se utiliza en la tabla 1.1), puesto que está lleno de ambigüedades y aspectos semánticos que pueden depender del contexto.

Artificial Por artificial entendemos lo que no ha evolucionado a partir del uso entre humanos, sino que ha sido creado expresamente, en este caso por ser usado con los ordenadores.

Para especificar las órdenes que debe seguir un ordenador lo que se utiliza es un lenguaje de programación. Se trata de un lenguaje artificial diseñado expresamente para crear algoritmos que puedan llevarse a cabo por el ordenador.

Al igual que hay muchas lenguas diferentes, también hay muchos lenguajes de programación, cada uno con sus propias características, que los hacen más

o menos indicados para resolver unos tipos de tareas u otros. Sin embargo, todos tienen una sintaxis muy definida, que hay que seguir para que el ordenador interprete correctamente cada orden que se le da. Es exactamente lo mismo que ocurre con las lenguas del mundo: para expresar los mismos conceptos, el catalán y el japonés usan palabras y normas de construcción gramatical totalmente diferentes entre sí.

En un lenguaje de programación determinado, la serie de órdenes concretas que se pide al ordenador que haga se llama conjunto de instrucciones.

Normalmente, el conjunto de instrucciones de un programa se almacena dentro de un conjunto de archivos. Estos archivos los edita el programador (usted) para crear o modificar el programa. Para los programas más sencillos es suficiente con un único archivo, pero para los más complejos puede ser necesario más de uno.

Los lenguajes de programación pueden clasificarse en diferentes categorías según sus características. De hecho, algunas de las propiedades del lenguaje de programación tienen importantes consecuencias sobre el proceso a seguir para poder crear un programa y para ejecutarlo. Hay dos formas de clasificar los lenguajes de programación:

Estas dos categorías no son mutuamente excluyentes.

- Según se trate de un lenguaje compilado o interpretado. Esta propiedad afecta a los pasos a seguir para obtener un archivo ejecutable. O sea, un archivo con el mismo formato que el de las aplicaciones que puede tener instaladas en su ordenador.
- Según se trate de un lenguaje de nivel alto o bajo. Esta propiedad indica el grado de abstracción del programa y si sus instrucciones están más o menos estrechamente vinculadas al funcionamiento del hardware de un ordenador.

### 1.3.1 ...en lenguaje de máquina

El lenguaje de máquina o código de máquina es el lenguaje que elegiríamos si quisiéramos realizar un programa que trabajara directamente sobre el procesador. Es interesante conocer porque ayuda a entender el proceso de generación de un programa.

En este lenguaje, cada una de las instrucciones se representa con una secuencia binaria, en ceros (0) y unos (1), y todo el conjunto de instrucciones del programa queda almacenado de forma consecutiva dentro de un archivo de datos en binario. Si intenta abrir con un editor de texto, lo que verá en pantalla son símbolos totalmente incomprensibles.

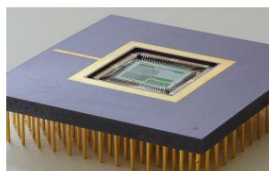
Cuando se solicita la ejecución de un programa en lenguaje de máquina, éste se carga en la memoria del ordenador. A continuación, el procesador va leyendo una por una cada una de las instrucciones, las decodifica y convierte en

#### Transistor

El transistor es el componente básico de un sistema digital. Puede considerarse como un interruptor, en el que 1 indica que pasa corriente y 0 que no pasa.

señales eléctricas de control sobre los elementos del ordenador para que realicen la tarea solicitada. A muy bajo nivel, casi puede llegar a establecerse una correspondencia entre los 0 y 1 de cada instrucción y el estado resultante de los transistores dentro de los chips internos del procesador.

El conjunto de instrucciones que es capaz de decodificar correctamente un procesador y convertir en señales de control es específico para cada modelo y está definido por su fabricante. El diseñador de cada procesador se inventó la sintaxis y las instrucciones del código máquina de acuerdo a sus necesidades cuando diseñó el hardware. Por tanto, las instrucciones en formato binario que puede decodificar un tipo de procesador pueden ser totalmente incompatibles con las que puede decodificar otro. Esto es lógico, ya que sus circuitos son diferentes y, por tanto, las señales eléctricas de control que debe generar son diferentes para cada caso. Dos secuencias de 0 y 1 iguales pueden tener efectos totalmente distintos en dos procesadores de modelos distintos, o resultar incomprensibles para alguno.



Como puede apreciarse, el procesador realmente es el centro neurálgico y el cerebro del ordenador, ya que otra de sus tareas es controlar la ejecución ordenada de cada instrucción. Imagen de mark.sze

Un programa escrito en lenguaje de máquina es específico para un tipo de procesador concreto. No se puede ejecutar sobre ningún otro procesador, a menos que sean compatibles. Un procesador concreto sólo entiende directamente el lenguaje de máquina especificado por su fabricante.

Aunque, como puede verse, en realidad existen muchos lenguajes de máquina diferentes, se usa esta terminología para englobarlos a todos. Si se desea concretar más se puede decir “lenguaje de máquina del procesador X”.

Ahora bien, estrictamente hablando, si optase por hacer un programa en lenguaje de máquina, nunca lo haría generando directamente archivos con secuencias binarias. Sólo hay que imaginar el aspecto de un programa de este tipo en formato impreso, consistente en una enorme retahíla de 0 y 1. Sería totalmente incomprensible y prácticamente imposible de analizar. En realidad lo que se usa es un sistema auxiliar de mnemotécnicos en el que se asigna a cada instrucción en binario un identificador en formato de texto legible, más fácilmente comprensible para los humanos. De esta forma, es posible generar un programa a partir de archivos en formato texto.

Esta recopilación de mnemotécnicos es lo que se conoce como el lenguaje ensamblador.

A título ilustrativo, la tabla [1.2](#) muestra las diferencias de aspecto entre un lenguaje de máquina y ensamblador equivalentes para un procesador de modelo 6502. Sin entrar en más detalles, es importante mencionar que tanto en lenguaje de máquina como en ensamblador cada una de las instrucciones se corresponde a una tarea muy simple sobre uno de sus componentes. Hacer que el ordenador realice tareas complejas implica tener que generar muchas instrucciones en estos lenguajes.

Tabla 1. 2. Equivalencia entre un lenguaje ensamblador y su lenguaje de máquina asociado

| Instrucción ensamblador | Lenguaje de máquina equivalente |
|-------------------------|---------------------------------|
| LDA n.º 6               | 1010100100000110                |
| CMP y 3500              | 110011010000000000110101        |
| LDA y 70                | 1010010101110000                |
| INX                     | 11101111                        |

1.3.2 ...mediante un lenguaje compilado

Para crear un programa lo que haremos es crear un archivo y escribir la serie de instrucciones que queremos que el ordenador ejecute. Para empezar será suficiente con un editor de texto simple.

Editores de texto simples Un editor de texto simple es el que permite escribir sólo texto sin formato. Son ejemplos Bloque de Notas (Windows), Gedit o Emacs (Unix).

Una vez acabado de escribir el programa, el conjunto de archivos de texto resultante donde se encuentran las instrucciones se dice que contiene el código fuente.

Este sistema de programar más cómodo para los humanos hace surgir un problema, y es que los archivos de código fuente no contienen lenguaje de máquina y, por tanto, resultan incomprensibles para el procesador. No se le puede pedir que lo ejecute directamente; esto sólo es posible usando lenguaje de máquina. Para poder generar código máquina es necesario realizar un proceso de traducción desde los mnemotécnicos que contiene cada archivo en las secuencias binarias que entiende el procesador.

El proceso llamado compilación es la traducción del código fuente de los archivos del programa en archivos en formato binario que contienen las instrucciones en un formato que el procesador puede entender. El contenido de estos archivos se llama código objeto. El programa que realiza este proceso se llama compilador.

Para el caso del ensamblador el proceso de compilación es muy sencillo, ya que es una mera traducción inmediata de cada mnemotécnico a la secuencia binaria que le corresponde. En principio, con esto ya sería suficiente para poder realizar cualquier programa, pero ceñirse sólo al uso de lenguaje ensamblador comporta ciertas ventajas e inconvenientes que hacen que en realidad no sea usado normalmente, sólo en casos muy concretos.

El código objeto de las instrucciones en la tabla 1.2 tiene este aspecto:  
101010010000011011  
001101000000000011  
010110100101011100  
0011101111

Por el lado positivo, con ensamblador el programador tiene control absoluto del hardware del ordenador a nivel muy bajo. Prácticamente se puede decir que controla cada señal eléctrica y los valores de los transistores dentro de los chips. Esto permite llegar a hacer programas muy eficientes en los que el ordenador hace exactamente lo que le decís sin ningún tipo de ambigüedad. En contraposición, los programas en ensamblador sólo funcionan para un tipo de procesador concreto, no son portables. Si se deben hacer para otra arquitectura, normalmente es necesario empezar de cero. Además —y es el motivo de mayor peso para pensárselo dos veces si se quiere usar este

lenguaje—crear un programa complejo requiere un grado enorme de experiencia sobre cómo funciona el hardware del procesador, y la longitud sería considerable.

Esto hace que sean programas complicados de entender y que sea necesario dedicar mucho tiempo a hacerlos.

## Lenguajes compilados de nivel alto

Actualmente, para generar la inmensa mayoría de programas se utilizan los llamados lenguajes de nivel alto. Éstos ofrecen un conjunto de instrucciones que son fáciles de entender para el ser humano y, por tanto, poseen un grado de abstracción más alto que el lenguaje ensamblador (ya que no están vinculados a un modelo de procesador concreto). Cada una de las instrucciones se corresponde a una orden genérica en la que lo más importante es su aspecto funcional (qué se quiere hacer), sin importar cómo se materializa esto en el hardware del ordenador ni mucho menos en señales eléctricas. En cualquier caso, es necesario advertir que esta clasificación no siempre es absoluta. Puede decirse que un lenguaje es de “nivel más alto o bajo que otro”, según el grado relativo de abstracción de sus instrucciones y su proximidad al funcionamiento interno del hardware de un ordenador.

El proceso para generar un programa a partir de un lenguaje de nivel alto es muy parecido al que se debe seguir para hacerlo usando el lenguaje ensamblador, ya que las instrucciones también se escriben en formato texto dentro de archivos de código fuente.

La ventaja adicional es que el formato y la sintaxis ya no están ligados al procesador, y por tanto, pueden tener el formato que desee el inventor del lenguaje sin que tenga que tener en cuenta el hardware de los ordenadores donde se ejecutará. Normalmente, las instrucciones y la sintaxis han sido elegidas para facilitar la tarea de creación y comprensión del código fuente del programa.

De hecho, en los lenguajes de nivel alto más frecuentes, entre ellos el que aprenderá a usar en este módulo, las instrucciones dentro de un programa se escriben como una secuencia de sentencias.

Una sentencia es el elemento mínimo de un lenguaje de programación, a menudo identificado por una cadena de texto especial, que sirve para describir exactamente una acción que el programa debe realizar.

Por tanto, a partir de ahora se usará el término sentencia en lugar de instrucción cuando el texto se refiera a un lenguaje de este tipo.

Una vez que se hayan acabado de generar los archivos con su código fuente, éstos también deben compilarse para traducirlos a código objeto. Ahora bien, en este caso, el proceso de traducción a código objeto será bastante más complicado que desde ensamblador.

El compilador de un lenguaje de alto nivel es un programa mucho más complejo.

En lo que se refiere al proceso de compilación, una consecuencia adicional de que el lenguaje no dependa directamente del tipo de procesador es que desde un mismo código fuente se puede generar código objeto para diferentes procesadores. Basta con disponer de un compilador diferente para cada tipo de procesador que desee soportarse. Por tanto, un mismo código fuente original puede servir para generar programas

Programas de nivel bajo

Se considera que el código de máquina y el ensamblador son los lenguajes de nivel más bajo existentes, ya que sus instrucciones dependen directamente del tipo de procesador.

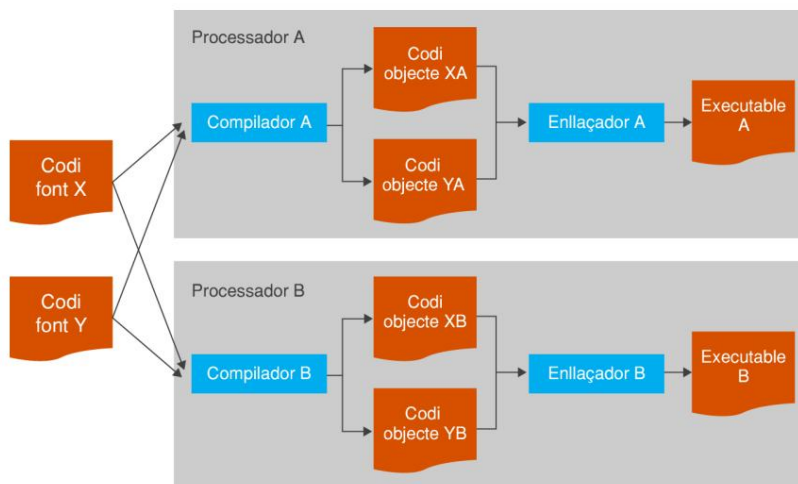
que funcionen con distintos tipos de procesador sin tener que modificarlo cada vez.

Puesto que para cada archivo de código fuente se genera un archivo de código objeto, después del proceso de compilación hay un paso adicional llamado enlazamiento (link), en el que estos dos códigos se combinan para generar un único archivo ejecutable.

Coloquialmente, cuando le pedimos que compile un programa ya se suele dar por hecho que también se enlazará, si procede. Sin embargo, formalmente se consideran dos pasos diferenciados.

La figura 1.2 muestra un esquema que sirve de resumen del proceso de generación del archivo ejecutable usando un lenguaje compilado.

Figura 1. 2. Proceso de compilación (y enlazamiento) del código fuente



Algunos ejemplos de lenguajes de nivel alto compilados muy populares son C o Pascal. Como se ha visto, el ensamblador también es un lenguaje compilado, pero de nivel bajo.

## Errores de compilación

El compilador es fundamental para generar un programa en lenguaje compilado, ya sea de nivel alto o bajo. Para poder realizar su trabajo de manera satisfactoria y generar código objeto a partir del código fuente es necesario que las instrucciones sigan perfectamente la sintaxis del lenguaje elegido. Por ejemplo, es necesario usar sólo las instrucciones especificadas en el lenguaje y hacerlo en el formato adecuado. Si no es así, el compilador es incapaz de entender el orden que se quiere dar al ordenador y no sabe cómo traducirlo a lenguaje máquina.

Cuando el compilador detecta que una parte del código fuente no sigue las normas del lenguaje, el proceso de compilación se interrumpe y anuncia que existe un error de compilación.

Cuando esto ocurra, habrá que repasar el código fuente e intentar averiguar dónde está el error. Normalmente, el compilador da algún mensaje sobre lo que considera que está mal.

Es necesario ser conscientes de que un programador puede llegar a dedicar buena parte del tiempo de la generación del programa a la resolución de errores de compilación. Ahora bien, que un programa compile correctamente sólo significa que se ha escrito de acuerdo con las normas del lenguaje de programación, pero no aporta ninguna garantía de que sea correcto, es decir, que haga correctamente la tarea para la que se ha ideado.

Los lenguajes de programación y el lenguaje natural

Intentando hacer un símil entre un lenguaje de programación y el lenguaje natural, si una persona que habla catalán es como un compilador, que es capaz de entender o traducir una frase siempre que se sigan las normas de esta lengua, sin algo de imaginación se le puede hacer difícil entender la frase: "helado kérem un comra". Hay palabras en un orden extraño, y además hay otras que no pertenecen al catalán o que no...

Por otra parte, la frase "El helado conduce una hoja de papel" puede ser gramaticalmente correcta y no tener ningún error de sintaxis. Alguien que hable catalán puede entenderla. Ahora bien, está claro que algo no encaja. En la comprensión del significado de un lenguaje existen aspectos que van más allá de la sintaxis, y los lenguajes de programación no son excepción.

### 1.3.3 ...mediante un lenguaje interpretado

En contraposición a los lenguajes compilados, tenemos los lenguajes interpretados. En este caso, no se hace una distinción interna entre nivel alto y bajo, puesto que la inmensa mayoría de lenguajes interpretados son de nivel alto. Lo que interesa es entender su idea general del funcionamiento y las diferencias con los compilados. Como en el caso de los lenguajes compilados, los programas también se escriben en archivos de texto que contienen código fuente. La divergencia surge inmediatamente después de terminar de escribirlos, en la forma en que se genera un archivo ejecutable. El quid de la cuestión es que, precisamente, ni se genera ningún código objeto ni ningún archivo ejecutable. Se trabaja directamente con el archivo de código fuente. Una vez éste está escrito, el proceso de creación del programa ejecutable ha finalizado.

Imagine un programa que acepta una serie de datos que codifican unas instrucciones, los va leyendo una por una y los va procesando de forma que actúa de una u otra forma, es decir, ejecuta una parte u otra de su propio código objeto según el tipo de instrucción leída. Al fin y al cabo, sería un programa que imita el comportamiento de un procesador, pero a escala de software. Pues esto es exactamente un intérprete.

Intérprete

Alerta, un intérprete no traduce el código fuente del programa a código objeto y entonces lo ejecuta. Lo que hace es ejecutar distintas instrucciones de su propio código según cada instrucción leída del código fuente.

Un lenguaje interpretado se ejecuta indirectamente, mediante la ayuda de un programa auxiliar llamado intérprete, que procesa el código fuente y gestiona su ejecución.

Al ser un intérprete un programa ejecutable está vinculado a un procesador ya un sistema operativo concretos.

Coloquialmente, la generación de bytecode a partir del código fuente se llama igualmente compilar.

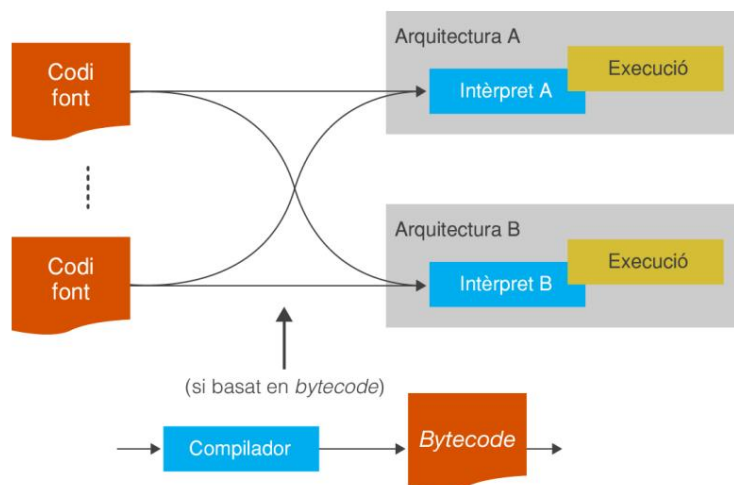
Al igual que en los lenguajes compilados, puede suceder que el programador haya incluido sin darse cuenta algún error de sintaxis en las instrucciones. En este caso, será el intérprete quien mostrará el error y se negará a ejecutar el programa hasta que haya estado solucionado.

Algunos lenguajes interpretados utilizan una aproximación híbrida. El código fuente se compila y como resultado se genera un archivo de datos binarios llamados bytecode. Sin embargo, este bytecode no es formalmente código objeto, ya que no es capaz de entenderlo el hardware de ningún procesador. Sólo un intérprete puede procesarlo y ejecutarlo. Simplemente es una forma de almacenar más eficiente y en menos espacio, en formato binario y no en texto, las instrucciones incluidas en el código fuente. Éste es el motivo por el cual, a pesar de necesitar un proceso de compilación, estos lenguajes no se consideran realmente compilados y se siguen clasificando como interpretados.

Por sus características, los lenguajes interpretados no requieren un proceso posterior de enlazamiento.

La figura 1.3 muestra un esquema del proceso de ejecución de un programa en lenguaje interpretado. Nótese que en el caso de un lenguaje con bytecode, lo que se proporciona al intérprete son archivos con la versión del código fuente previamente compilado en bytecode, y no el código fuente directamente.

Figura 1. 3. Proceso de interpretación del código fuente



El término "arquitectura" se refiere a la combinación de procesador y sistema operativo

Entre los lenguajes interpretados más conocidos encontramos JavaScript, PHP o Perl.

Muchos son lenguajes de script, que permiten el control de aplicaciones dentro de un sistema operativo, llevar a cabo procesos por lotes (batch) o generar dinámicamente contenido web.

Entre los lenguajes interpretados basados en bytecode, Java es uno de los más populares.

#### 1.3.4 Entornos integrados de edición

Una vez se ha descrito el proceso general para desarrollar y llegar a ejecutar un programa, se hace evidente que es necesario tener instalados y correctamente configurados dos programas completamente diferentes e independientes en su ordenador para desarrollarlos: editor, por un lado, y compilador (incluyendo el enlazador) o intérprete por otro, según el tipo de lenguaje.

Cada vez que desee

modificar y probar su programa tendrá que ir alternando ejecuciones entre los dos. Realmente, sería mucho más cómodo si todo pudiera hacerse desde un único programa, que integrara a los tres. Un editor avanzado desde el que se pueda compilar, enlazar en su caso, e iniciar la ejecución de código fuente para comprobar si funciona.

#### Ejemplos de IDE

Algunos ejemplos de IDE son Visual Studio, para los lenguajes C#, C++ y Visual Basic; Netbeans, para los lenguajes Java y Ruby; el Dev-Pascal, para el lenguaje Pascal, o el Dev-C, para el lenguaje C.

Un IDE ( integrated development environment o entorno integrado de desarrollo) es una herramienta que integra todo lo necesario para generar programas de ordenador, de modo que el trabajo sea mucho más cómodo.

La utilización de estas herramientas agiliza increíblemente el trabajo del programador. Además, los IDE más modernos van más allá de integrar editor, compilador y enlazador o intérprete, aportando otras características que hacen aún más eficiente la tarea de programar. Por ejemplo:

- Posibilidad de resaltar con códigos de colores los diferentes tipos de instrucciones o aspectos relevantes de la sintaxis del lenguaje soportado, para facilitar la comprensión del código fuente.
- Acceso a documentación y ayuda contextual sobre las instrucciones y sintaxis de los lenguajes soportados.
- Detección, y en algunos casos incluso corrección, automática de errores de sintaxis en el código, de forma similar a un procesador de texto. Así, no es necesario compilar para saber que el programa está mal.
- Apoyo simultáneo del desarrollo de lenguajes de programación diferentes.
- Un depurador, una herramienta muy útil que permite pausar la ejecución del programa en cualquier momento o realizarla instrucción por instrucción, de forma que permite analizar cómo funciona el programa y detectar errores.
- En los más avanzados, sistemas de ayuda para la creación de interfaces gráficas.

En definitiva, usar un IDE para desarrollar programas es una opción muy recomendable. Sin embargo, hay que tener presente que son programas más complejos que un simple editor de texto y, como ocurriría con cualquier otro programa, es necesario dedicar cierto tiempo a familiarizarse con éstos y con las opciones de que disponen.

### 1.4 Su primer programa

Hay distintos lenguajes de programación, algunos realmente muy distintos entre sí. Antes de seguir adelante hay que elegir uno que será el usado para practicar todos los conceptos de programación básica que verá de ahora en adelante. Una vez que aprenda a programar en un lenguaje, dominar otros lenguajes le será muy fácil, ya que muchos de los conceptos básicos, e incluso algunos aspectos de la sintaxis, se mantienen entre diferentes lenguajes de nivel alto.

En este módulo, el lenguaje con el que aprenderemos a programar será Java.

#### 1.4.1 Características relevantes del Java

Desgraciadamente, no existe el lenguaje perfecto, sin ningún inconveniente y que sea ideal para crear cualquier tipo de programa. Siempre es necesario llegar a un compromiso entre ventajas e inconvenientes. De hecho, la elección misma de qué lenguaje es necesario usar puede llegar a condicionar enormemente el proceso de creación de un programa, y no es sensato usar siempre el mismo para resolver cualquier problema. En cualquier caso, vale la pena comentar los motivos por los que se considera interesante usar Java.

- **Popular:** Java fue creado en 1995 por la firma Sun Microsystems, que en 2009 fue comprada por la empresa de bases de datos Oracle. Su propósito era ofrecer un lenguaje lo menos ligado posible a la arquitectura sobre la que se ejecuta. Esto le convirtió en sus inicios en el mecanismo más versátil existente para ejecutar aplicaciones sobre navegadores web (actualmente, la tecnología Flash ya le ha tomado el relevo). Desde entonces, su popularidad ha ido en aumento también como lenguaje para crear aplicaciones de escritorio, y actualmente es uno de los lenguajes más utilizados en este campo. Esto hace que la demanda de profesionales que lo dominen sea muy alta y que tenga una gran aceptación y cantidad de documentación disponible.
- **De nivel alto con estricto compilador:** Java es un lenguaje de nivel bastante alto, con todas las ventajas que ello implica. Adicionalmente, su compilador es especialmente estricto a la hora de realizar comprobaciones sobre la sintaxis empleada y cómo se manipulan los datos que se están tratando en el programa. Si bien esto a veces puede parecer algo enfadador cuando se producen ciertos errores de compilación, en realidad es una ventaja, ya que enseña al programador a tener más grado de control sobre el código fuente que genera, de modo que sea correcto.
- **Multiplataforma:** uno de los factores decisivos en la popularidad de Java es que sus programas se pueden ejecutar en cualquier plataforma sin necesidad de volver a compilar. Una vez que el código fuente se ha compilado una vez, el bytecode resultante puede ser llevado a otras plataformas basadas en otros tipos de procesador y seguirá funcionando. Basta con disponer del intérprete correspondiente para la nueva plataforma. Esto homogeneiza enormemente el aprendizaje del lenguaje independientemente de la plataforma que está utilizando para estudiarlo.
- **Orientado a objetos:** éste es el nombre de una metodología avanzada, muy popular y útil, para diseñar programas. Si bien sus detalles están totalmente fuera del alcance de este módulo, basta con decir que es un mecanismo de

Aplicación de escritorio Se considera una aplicación de escritorio (desktop application) la que es totalmente autocontenida y que puede ejecutarse en ordenadores de sobremesa o portátiles. Este término se utiliza en contraposición con las aplicaciones basadas en Web.

nivel muy alto para acercar la forma en que se hacen los programas al método de pensamiento humano. En este curso no veremos un enfoque orientado a objetos de la programación, pero el lenguaje Java le permitirá, cuando conozca esta tecnología, trabajar con ella.

También habrá que ser conscientes de algunas particularidades:

- Orientado a objetos: algunas partes de la sintaxis y la nomenclatura formal de Java están íntimamente vinculadas a la metodología de la orientación a objetos y no pueden separarse de ellos. Por tanto, en algunos momentos será inevitable tener que hacer frente a aspectos ligados a la orientación a objetos, aunque se trate de una metodología avanzada que no sea objeto de estudio en este curso.  
La ventaja es que cuando pase a programar con orientación a objetos ya conocerá el lenguaje.
- Interpretado con bytecódigo: ésta es una característica derivada de que sea multiplataforma.  
Al tratarse de un lenguaje interpretado, es necesario disponer del intérprete correctamente instalado y configurado en cada máquina donde desee ejecutar su programa. Esto significa que hay un programa más que debe configurar correctamente en su sistema. Esto también hace que la ejecución de los programas en Java no siga el proceso típico de cualquier otra aplicación (por ejemplo, ejecutarlo desde línea de mandatos o hacer doble clic en la interfaz gráfica). No hay ningún archivo que pueda identificarse claramente como ejecutable.

#### 1.4.2 Creación y ejecución de programas Java

Este apartado se centra en mostrar detalladamente cómo se crea y ejecuta un programa en lenguaje Java.

Dado que Java es un lenguaje interpretado, las herramientas que necesitan son:

- Un editor de texto simple cualquiera.
- Un compilador del llenguatge Java, per generar bytecode.
- Un intérprete de Java, para poder ejecutar los programas.

Disponer de un editor de texto es el menos problemático, ya que todos los sistemas operativos de propósito general suelen tener instalado alguno por defecto. Ahora bien, cuando se edita un archivo de código fuente, es necesario asignarle una extensión específica de acuerdo con el lenguaje de programación empleado, de modo que pueda ser fácilmente identificado como tal.

La extensión de los archivos de código fuente en Java es la .java.

Por ahora se tratará el caso de la creación de programas sencillos que se compongan de un único archivo de código fuente.

Otros lenguajes tienen otras extensiones. A título de ejemplo, en lenguaje C los archivos tienen la extensión `.c`, en ensamblador `.asm`, en Perl `.pl`, etc. Es importante que al editar código fuente desde cualquier editor de texto guarde el archivo con esta extensión y no la que le ofrezca por defecto (normalmente, `.txt`).

¿Por qué se llama notación de camello?

En el caso concreto de Java, existe una convención a la hora de dar nombre a un archivo de código fuente. Se suele usar UpperCamelCase (notación de camello con mayúsculas). Ésta corresponde a usar sólo letras consecutivas sin acentos (ni espacios, subrayados o números), y en los que la inicial de cada palabra usada sea siempre en mayúscula. Esto no es estrictamente imprescindible, pero sí muy recomendable, ya que es el estilo de nomenclatura que siguen todos los programadores de Java y la que encontrará en documentación, guías u otros programas. Además, en algunos sistemas, el uso de caracteres especiales, como los acentos, puede llevar a errores de compilación.

Algunos ejemplos de nombres de archivos de código fuente aceptables son: `Prueba.java`, `HolaMon.java`, `MiPrograma.java`, etc.



Notación de camello

El compilador y el intérprete de Java son dos programas que tendrá que instalar en su ordenador, ya que normalmente no están preinstalados por defecto. Existen varios tipos, de distintos fabricantes, pero lo más recomendable es instalar lo que proporciona de forma gratuita el actual propietario de Java, la empresa Oracle, en su página de descargas. Existen diferentes versiones del entorno de trabajo con Java, pero lo habitual es trabajar con la Java SE (Java standard edition, edición estándar de Java).

La página de descargas de Oracle es

<http://www.oracle.com/downloads>. Existen versiones de JDK para diferentes procesadores y sistemas operativos.

El compilador de Java está incluido dentro del llamado JDK (Java development kit, equipo de desarrollo de Java). Éste proporciona una serie de ejecutables vía línea de mandatos que sirven para realizar diferentes tareas con código fuente Java.

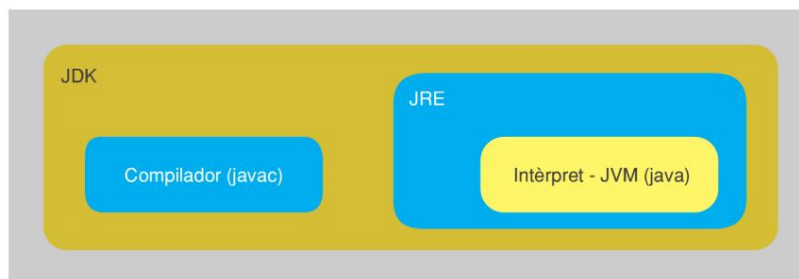
El programa que pone en marcha el compilador es el ejecutable llamado `javac` (en un sistema Windows, `javac.exe`). Por tanto, para el archivo con código fuente Java llamado `HolaMon.java` habría que abrir una línea de instrucciones y ejecutar:

```
1 javac HolaMon.java
```

El archivo con bytecode resultante del proceso de compilación se llama igual que el archivo de código fuente, pero con la extensión `.class`.

Una vez disponemos del archivo con bytecode, éste sólo puede ser ejecutado con la ayuda del intérprete de Java, conocido popularmente como la JVM (Java virtual machine, máquina virtual de Java). Ésta se incluye dentro del paquete llamado JRE (Java runtime environment, entorno de ejecución de Java). Asimismo, el JRE ya va incorporado automáticamente dentro del JDK. La figura 1.4 muestra un esquema de la relación entre herramientas incluidas en los distintos paquetes para el desarrollo y ejecución de programas en Java.

Figura 1. 4. Relación de herramientas incluidas en cada paquete de software de Java



El ejecutable que pone en marcha el intérprete de Java es el llamado java (en un sistema Windows, java.exe). Una vez que se dispone del archivo de bytecode HolaMon.class se puede ejecutar desde la línea de mandatos haciendo:

```
1 java HolaMon
```

Nótese que no se especifica ninguna extensión. Él solo deduce que la extensión debe ser .class. Inmediatamente, el programa se pondrá en marcha.

Aunque este es el entorno básico de desarrollo de Java, afortunadamente existen varios IDE que soportan este lenguaje y concentran en un único entorno este software. Para trabajar con Java existen varios IDE con diferentes grados de popularidad: Netbeans, Eclipse, Jcreator, etc. Cada uno tiene sus propias particularidades y grados de complejidad de configuración y uso. Sin embargo, hay que decir que actualmente el IDE oficial de Oracle es el Netbeans.

Dispone de un anexo en el que se explica el funcionamiento del IDE que va a usar a lo largo del curso.

Por último, es importante remarcar que en caso de que sólo desee ejecutar un programa que ya ha sido desarrollado por alguien, sea mediante un IDE o no, la única herramienta que necesita tener instalada en el ordenador para poder ejecutar el archivo .class resultante es la última versión de JRE. No es necesario instalar nada más.

### 1.4.3 ¡Hola, mundo!

Dentro del ámbito de la programación es tradición que el primer programa que se escribe y ejecuta cuando se inicia el estudio de un nuevo lenguaje sea el llamado “¡Hola, mundo!” (originalmente en inglés, “Hello, world!”). Esta tarea es un simple ejercicio de copiar el código fuente del programa, por lo que ni siquiera es necesario entender aún la sintaxis del lenguaje. El objetivo principal de realizar este programa es ver que el entorno de trabajo se encuentra correctamente instalado y configurado, ya que por su sencillez es difícil que dé problemas. Además, también le sirve como plantilla de la estructura básica de un programa en el lenguaje escogido y permite repasar su estructura y algunos de los elementos básicos.

El código fuente para la versión en Java es el siguiente:

```
1 //El programa "¡Hola, mundo!" en Java
```

Puede encontrar una lista del código fuente del programa “Hello, world!” para distintos lenguajes en <http://www.scriptol.com/programming/hello-world.php>

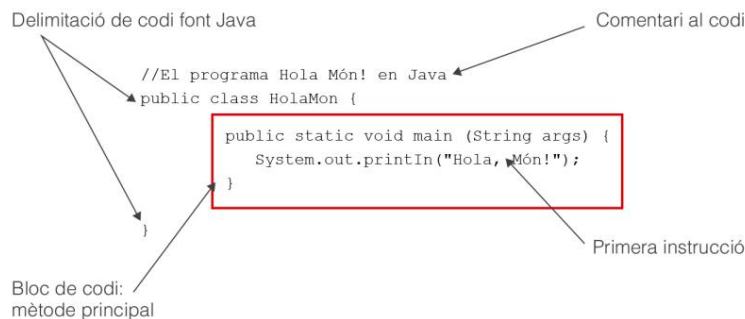
```
2 public class HolaMon { public static void  
3     main(String[] args) { System.out.println("Hola, món!");  
4  
5     }  
6 }
```

Reto 1: copie el código fuente de este programa en un archivo llamado HolaMon.java y ejecútelo en su entorno de trabajo. El resultado de la ejecución debería ser que por pantalla se muestre la frase Hola, món!.

El lenguaje Java es sensible a mayúsculas y minúsculas. El significado de los términos varía según están escritos. Por tanto, es imprescindible que tanto el código fuente como el nombre del archivo se escriban exactamente tal y como se muestran.

La figura 1.5 le muestra a qué parte característica de un archivo de código fuente se corresponde cada parte del texto del código fuente.

Figura 1.5. Elementos del código fuente del programa "¡Hola, mundo!"



#### Importación de extensiones (bibliotecas)

Una biblioteca es un conjunto de extensiones en el conjunto de instrucciones disponibles cuando genera un programa. Para poder utilizar estas instrucciones adicionales es necesario que, dentro del código fuente, como preámbulo, declare su importación. De lo contrario, las extensiones no están disponibles de forma predeterminada dentro del lenguaje. En el caso del programa "¡Hola, mundo!", al ser muy sencillo, no es necesario, pero en Java se usaría la sintaxis:

```
1 importar <nomBiblioteca>;
```

#### Indicador de inicio del código fuente

El código fuente donde comienza realmente el programa en Java comienza con el texto de declaración que se muestra a continuación, en el que `<Nombre Archivo>` puede variar pero siempre debe corresponder exactamente con el nombre del archivo que lo contiene. Esto es esencial. Por otra parte, el compilador nos dará un error. El código del programa se escribirá a continuación, siempre entre claves, `{...}`.

```
1 clase pública <NomFiber> {  
2     ...  
3 }
```

Este texto declara que este archivo es el inicio de lo que en nomenclatura Java se llama una clase. Éste es un término estrechamente vinculado a la orientación a objetos, pero por ahora lo usaremos simplemente para referirnos a un archivo que contiene código fuente de Java.

Para desgracia del programador novato, muchas partes de la sintaxis de Java están vinculadas a la orientación a objetos.

### Comentarios al código

Opcionalmente, también se pueden escribir comentarios dentro del código fuente. Se trata de texto que representa anotaciones libres en el programa, pero que el compilador no procesa. Normalmente, un comentario se identifica por ser una línea de texto libre que comienza por una combinación de caracteres especiales. Los comentarios pueden estar en cualquier parte del archivo, ya que el compilador los ignora. No sirven realmente para nada de cara a la ejecución del programa, y sólo son de utilidad para quien está editando el archivo de código fuente.

Comentar el código fuente para explicar qué hace cada parte del programa, especialmente en aquellas más complejas, es una tarea muy importante que demuestra si un programador es cuidadoso o no.

En el programa Hola, mundo está el siguiente comentario:

```
1 //El programa "¡Hola, mundo!" en Java
```

En lenguaje Java los comentarios se escriben o bien precediéndolos con dos barras, en caso de que tengan una sola línea, o bien en el formato siguiente si ocupan más de una línea.

```
1 /** *  
2     Este es el programa "Hola, mundo!" * en el lenguaje de  
3     programación Java 4 **/
```

A partir de ahora, en todos los ejemplos de código, los detalles sobre lo que hace cada parte del programa se describirán mediante un comentario antes de cada instrucción.

### Indicador de la primera instrucción para ejecutar

Para que el ordenador sepa por dónde empezar a ejecutar instrucciones, ante todo debe saber cuál es la primera de todas. Una vez localizada, continuará ejecutando el resto de forma secuencial, por orden de aparición en el código fuente. En algunos lenguajes esto se hace implícitamente, puesto que la primera instrucción es directamente la primera línea de texto que aparece en el código fuente. En el caso de Java, existe un texto que la indica claramente.

En Java, el bloque de instrucciones en el que se engloba la primera instrucción del programa en la mayoría de lenguajes de programación se llama el método principal.

Este método principal engloba todas las instrucciones del programa dentro de un bloque de instrucciones, entre claves, {...}. Antes de las claves hay una serie de texto que debe escribirse exactamente tal y como se muestra. Si no, el intérprete de Java será incapaz de encontrarlo y de iniciar correctamente la ejecución del programa.

Concretamente, dirá que "No encuentra el método principal" (main method not found).

```
1 public static void main (String[] args) {  
2     ...  
3 }
```

La primera instrucción es la primera escrita justo después de la clave abierta, {. La última instrucción es la escrita inmediatamente antes de la clave cerrada, }.

#### Bloques de código o instrucciones

Las instrucciones o sentencias del programa están escritas una detrás de otra, normalmente en líneas separadas para hacer el código más fácil de entender. En algunos lenguajes, al final de cada línea es necesario un delimitador especial, que sirva para indicar cuándo termina una sentencia y comienza otra. El salto de línea no es suficiente. En el caso del Java, se trata del punto y coma, ;.

Los programas más simples, como los que veremos por ahora, sólo disponen de un único bloque de instrucciones.

Las diferentes instrucciones suelen agruparse en bloques de instrucciones. El inicio y el fin de cada bloque distinto quedan identificados en Java porque las instrucciones están rodeadas por claves, {...}.

Por tanto, en este programa sólo hay una única, a la vez primera y última, instrucción.

```
1 System.out.println("¡Hola, hermano!");
```

Aunque todavía no conoce ninguna de las instrucciones de Java ni su sintaxis, posiblemente puede deducir de todas formas que ésta sirve para ordenar al ordenador que muestre por pantalla el texto escrito entre comillas. Es decir, que es un comando sobre el componente de entrada/salida (pantalla). Ésta es una instrucción muy útil. En este código también se puede apreciar que en el lenguaje Java las instrucciones terminan con un punto y coma, ;.

Reto 2: modifique el código fuente para que el programa escriba por pantalla ¡Adiós, mundo!.



## 2. Manipulación básica de datos

El propósito principal de todo programa de ordenador, en última instancia, es procesar datos de todo tipo. Para lograr esta tarea, el ordenador debe almacenar los datos en la memoria, de forma que posteriormente el procesador pueda leerlos y transformarlos de acuerdo con los propósitos del programa. Por tanto, un conjunto de instrucciones muy importante dentro de todo lenguaje de programación, y un buen lugar por donde empezar el aprendizaje, es el que da a la memoria las órdenes de guardarlas o consultarlas. Una vez que ya se saben gestionar datos en la memoria, es el momento de ver algunas de las transformaciones básicas que permite realizar el procesador.

El término dato indica toda información que utiliza el ordenador en las ejecuciones de los programas.

Aunque en ocasiones se usa el término datos, en plural, como si fuera algo en general, hay que tener en cuenta que dentro de un programa cada dato que se quiere tratar es un elemento individual e independiente. Por ejemplo, en un programa que suma dos números cualesquiera existen tres datos con los que trabaja: los dos operandos iniciales y el resultado final.

Por otra parte, hasta el momento se ha dicho que el ordenador se encarga de procesar y transformar cada uno de estos datos individuales, pero tampoco se ha especificado exactamente cómo se representan o clasifican los diferentes tipos de datos dentro del programa, o de qué forma se pueden transformar. Ha llegado el momento de concretar algo más todos estos detalles.

### 2.1 Tipo de datos

Estrictamente hablando, cualquier información puede transformarse en datos que pueda entender y manipular un ordenador. Un dato individual dentro de su programa puede ser un documento de texto, una imagen, un plano de un edificio, una canción, etc.

Basta con ver la inmensa variedad de programas que existen hoy en día para hacerse una idea. Ahora bien, uno de los puntos importantes es que dentro de un programa los datos pueden clasificarse dentro de diferentes categorías, los tipos de datos.

Un tipo de dato es la definición del conjunto de valores válidos que pueden tomar unos datos y el conjunto de transformaciones que se pueden realizar.

Por ejemplo, en su día a día a menudo trata con datos que tienen como característica común el hecho de que se pueden representar mediante números: una

distancia, una edad, un período de tiempo, etc. Puede decirse que una ciudad está a 8 km de otra, que alguien tiene 30 años o que han pasado 15 días desde un evento. Por tanto, los valores 8, 30 o 15 formarían parte de un mismo tipo de dato.

También como ejemplo, en contraposición a los datos de tipo numérico, otra información que trata habitualmente es texto en palabras: el nombre de una mascota, de una calle, de una institución, etc. En este caso, puede estar hablando de “Shiro”, del “Paseo de Gracia”, del “Institut Obert de Catalunya”, etc. En este caso, los datos se representan con distintos símbolos y expresan otras ideas. Por tanto, formarían parte de otro tipo de dato.

Una propiedad importante para ver si dos datos son de distintos tipos es si el conjunto de valores con el que se pueden expresar también es diferente. En los dos ejemplos anteriores, la consideración de que se trata de tipos de datos diferentes también se encuentra soportada por este hecho. Por ejemplo, el conjunto de valores que puede tomar el nombre de una calle o una persona no es cualquiera, puesto que no se puede expresar exclusivamente mediante valores numéricos.

Como cada dato dentro de su programa siempre debe pertenecer a algún tipo, precisamente parte de su tarea como programadores es identificar cuál es el tipo que se acerca más a la información que desea representar y procesar. Establecer cuál es el tipo de un dato implica que se establecen un conjunto de condiciones sobre ese dato a lo largo de todo el programa. Una de las más importantes es el efecto sobre la forma en que este dato se representará, tanto internamente dentro del hardware del ordenador como a la hora de representarlo en el código fuente de sus programas. Recuerde que, por su naturaleza de sistema digital, todos los datos que hay dentro de un ordenador se codifican como secuencias binarias.

Qué secuencia de 0 y 1 es necesario usar para representar el valor de cada dato dependerá del tipo escogido.

Cada lenguaje de programación incorpora sus propios tipos de datos y, aparte, casi siempre ofrece mecanismos para definir otros tipos partiendo de tipos de datos ya existentes. Por tanto, como programadores, sólo tiene que elegir entre todos los que ofrece el lenguaje cuál se acerca más al tipo de información que desea tratar.

Sin embargo, desgraciadamente no se puede garantizar que las instrucciones básicas definidas en la sintaxis de un lenguaje de programación sean capaces de tratar de manera directa cualquier dato. Esto sólo sucede con un conjunto limitado de tipos de datos muy simples, llamados tipos primitivos.

Los tipos primitivos de datos son los que ya están incorporados directamente dentro de un lenguaje de programación, y son usados como piezas básicas para construir otros complejos.

---

Los lenguajes de programación identifican cada tipo con una palabra clave propia.

---

El soporte a diferentes tipos primitivos puede variar también entre lenguajes. De todas formas, hay cuatro que se puede considerar que, de una u otra forma, todos los lenguajes los soportan. El motivo es que estos tipos están estrechamente ligados a los tipos de secuencias binarias que normalmente un ordenador puede procesar y representar directamente dentro de su hardware. Se trata de números enteros, reales, caracteres y booleanos.

Para empezar, basta con ver qué tipo de datos representan. Ahora bien, para ilustrar cómo podemos usar dentro del código fuente de un programa datos que pertenecen a tipos primitivos, empezará trabajando con literales.

Un literal es un texto usado para representar un valor fijo dentro del código fuente de un programa.

### 2.1.1 El tipo de dato booleano

El tipo de dato booleano representa un valor de tipo lógico a fin de establecer la certeza o falsedad de un estado o afirmación.

La palabra clave para identificar este tipo de datos en Java es `boolean`.

- Ejemplos de literales de un dato booleano: `true` (cierto) o `false` (falso). No hay otro.
- Ejemplos de datos que suelen representarse con un booleano: interruptor encendido o apagado, estar casado, tener derecho de voto, disponer de carnet de conducir B1, la contraseña es correcta, etc.

Un literal de tipo booleano se representa simplemente escribiendo el texto tal y como se ha descrito arriba. En un IDE, normalmente este texto queda resaltado en un color especial para que quede claro que se ha escrito un literal de tipo booleano. Por ejemplo, el siguiente programa en Java muestra una serie de literales de este tipo por pantalla.

Pruébalo en su entorno de trabajo. Recuerde que debe estar dentro de un archivo llamado `LiteralsBoolea`, y que el nombre no puede tener acentos.

```
1 public class LiteralsBoolea { public static void main
2     (String[] args) { System.out.println(true); System.out.println(false);
3
4
5     }
6 }
```

### 2.1.2 El tipo de dato entero

El tipo de datos introducido representa un valor numérico, positivo o negativo, sin decimales.

La palabra clave para identificar este tipo de dato en Java es `int`.

- Ejemplos de literales enteros: `3`, `0`, `-345`, `138764`, `-345002`, etc.
- Ejemplos de datos que suelen representarse con un entero: edad, día del mes, año, número de hijos, etc.

Un literal de tipo entero se representa simplemente escribiendo un número sin decimales. Por ejemplo, el siguiente programa en Java muestra una serie de literales de este tipo por pantalla. Pruébalo en su entorno de trabajo. Recuerde que debe estar dentro de un archivo llamado `LiteralsEnter`.

```
1 clase pública LiteralsEnter { public static void main (String[]  
2     args) {  
3         System.out.println(3);  
4         System.out.println(0);  
5         System.out.println(-345);  
6         System.out.println(138764);  
7         System.out.println(-345002);  
8     }  
9 }
```

### 2.1.3 El tipo de dato real

La palabra clave para identificar este tipo de dato en Java es `double`.

El tipo de datos real representa un valor numérico, positivo o negativo, con decimales.

- Ejemplos de literales reales: 2.25, 4.0, -9653.3333, 100.0003, etc.
- Ejemplos de datos que suelen representarse con un real: un precio en euros, el récord mundial de los 100 m lisos, la distancia entre dos ciudades, etc.

Para referirse a un dato de tipo real, éste incluye siempre sus decimales con un punto (.). En los ejemplos note el detalle del valor 4.0. Los números reales representan valores numéricos con decimales, pero en su definición nada impide que el decimal sea 0. Por tanto, estrictamente, el valor 4 y el valor 4.0 corresponden a tipos de datos diferentes. El primero es un valor para un tipo de dato entero y el segundo para uno real.

Por ejemplo, el siguiente programa en Java muestra una serie de literales de este tipo por pantalla. Pruébalo en su entorno de trabajo. Recuerde que debe estar dentro de un archivo llamado `LiteralsReal`.

```
1 public class LiteralsReal { public static void  
2     main (String[] args) { System.out.println(2.25); System.out.println(4.0);  
3         System.out.println(-9653.3333);  
4         System.out.println(100.0003);  
5  
6  
7     }  
8 }
```

Si desea saber más sobre las diferencias entre la representación de enteros y reales, puede buscar información sobre los formatos "Complemento a Dos" (Two's Complement) y "IEEE 754".

Una pregunta que quizás se puede plantear ahora es qué sentido tiene el tipo entero si con el tipo real ya podemos representar cualquier número, con decimales incluso. ¿No es algo redundante? Bien, de entrada puede parecer que sí, pero hay un motivo para diferenciarlos. Sin entrar en detalles muy técnicos, representar y realizar operaciones con enteros internamente dentro del ordenador (en binario) es mucho más sencillo y rápido que

no con reales. Además, un entero requiere menos memoria. Evidentemente, un programa no será perceptiblemente más lento o más rápido por el simple hecho de realizar una operación entre dos datos de tipo real en lugar de entero, pero es una buena costumbre usar siempre el tipo de dato que se adapte exactamente a sus necesidades. Si un dato no tiene sentido que tenga decimales, como un número de año o mes, es mejor usar el tipo entero.

#### 2.1.4 El tipo de dato carácter

El tipo de dato carácter representa una unidad fundamental de texto usada en cualquier alfabeto, número o signo de puntuación o exclamación.

La palabra clave para identificar este tipo de dato en Java es `char`.

- Ejemplos de literales de caracteres: `'a'`, `'A'`, `'4'`, `'>'`, `'?'`, `'Γ'` (letra griega gamma) mayúscula), etc.
- Ejemplos de datos que suelen representarse con un carácter: cada uno de los símbolos individuales de un alfabeto.

Para referirse a un dato de tipo carácter, éste se rodea de comillas simples (`'`). Por tanto, no es lo mismo el carácter `'4'` y que el valor entero `4`, ya que pertenecen a tipos diferentes. El primero lo usará para referirse a una representación textual, mientras que el segundo es el concepto propiamente matemático. Tenga cuidado, ya que entre las comillas simples sólo puede haber un solo carácter, o Java dirá que la sintaxis no es correcta.

Por ejemplo, el siguiente programa en Java muestra una serie de literales de este tipo por pantalla. Pruébalo en su entorno de trabajo. Recuerde que debe estar dentro de un archivo llamado `LiteralsCaracter`, sin usar ningún acento en el nombre.

```
1 public class LiteralsCaracter { public static void main (String[] args)
2     { System.out.println("a"); System.out.println("A"); System.out.println("4");
3       System.out.println(">"); System.out.println("?");
4       System.out.println("Γ");
5
6
7
8
9     }
10 }
```

Java representa sus caracteres internamente usando la tabla UNICODE.

#### Sistemas de representación de los caracteres

A la hora de decidir cómo se representa internamente un carácter, se utilizan diferentes tablas de códigos establecidos, entre los que destacan los códigos ASCII y UNICODE. El código ASCII fue uno de los más extendidos, sobre todo por la gran proliferación de ordenadores personales (llamados en inglés personal computer, PC). El código ASCII inicial representaba 128 caracteres en lugar de 256. Cuando se necesitaron caracteres especiales para los distintos idiomas, se amplió con 128 caracteres más, constituyéndose el código ASCII extendido.

Desgraciadamente, el código ASCII tiene una pega, y es que sólo permite representar alfabetos occidentales, por lo que los programas que lo usan para representar sus datos

de tipo carácter, son incompatibles con sistemas con otros alfabetos, como todos los asiáticos, el cirílico, etc. Por este motivo, posteriormente se creó la tabla de codificación UNICODE, que permite codificar hasta 65.536 caracteres pero manteniendo la compatibilidad con la codificación ASCII. Esto permite apoyar cualquier lengua actual, e incluso antiguas, como los jeroglíficos egipcios. Este sistema es el que actualmente utilizan la mayoría de aplicaciones modernas.

## 2.2 Transformación de los datos

Dentro de un programa los datos se terminarán representando usando alguno de los cuatro tipos primitivos. Una vez que haya elegido qué tipo utilizará para representar una información concreta, ya ha limitado el conjunto de valores que puede tomar. Ahora bien, otro punto muy importante de esta decisión es que también habrá fijado cómo es posible transformarlas. Para cada tipo de dato, sus valores sólo pueden transformarse de unas maneras muy concretas: mediante lo que formalmente se conoce como operaciones. Cada tipo de dato establece el conjunto de operaciones admisibles sobre los valores que comprende.

---

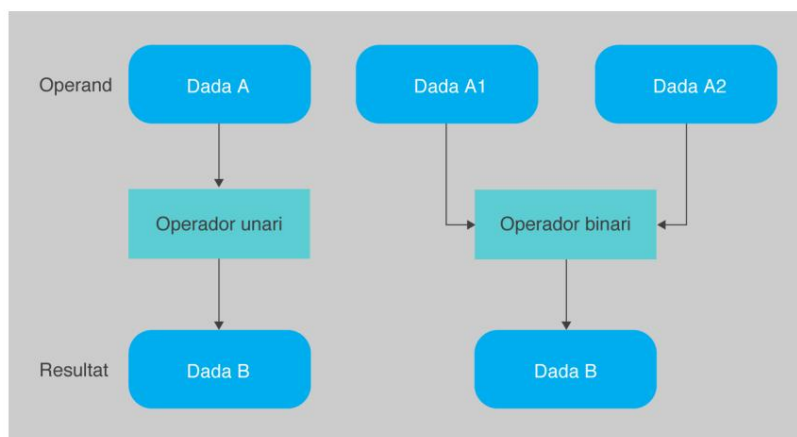
Un lenguaje de programación que sólo permite operaciones entre datos del mismo tipo se llama fuertemente tipado. Java lo es.

---

Normalmente, sólo se pueden realizar operaciones entre datos que pertenezcan al mismo tipo. Como suele decirse, no se pueden sumar manzanas y naranjas. Ahora bien, sí puede darse el caso de que una operación entre dos tipos de datos iguales dé como resultado un dato de un tipo diferente. Por ejemplo, no podrá sumar un número entero (el número 3, por ejemplo) con un booleano (el valor true, por ejemplo). De entrada, no tendría ningún sentido.

Aplicar operaciones sobre varios datos le permite crear nuevos valores. Las operaciones se dividen en dos tipos: unarias o binarias, según el número de operandos que usan (uno o dos). La figura 2.1 presenta un esquema de cómo las operaciones nos permiten crear nuevos datos.

Figura 2. 1. Esquema de aplicación de operadores y operandos para generar nuevos datos



El símbolo de cómo se identifica cada una de las operaciones dentro de un tipo de dato es su operador. Los datos sobre los que se aplica una operación son sus operandos.

### 2.2.1 Operaciones entre booleanos

Este tipo de dato debe su nombre al álgebra de Boole, que es el conjunto de normas y operaciones que rigen cómo se pueden combinar los valores verdadero o falso.

Las tablas de verdad son una representación que nos permite averiguar, con claridad y fiabilidad, todos los resultados posibles de una operación a partir de todas combinaciones de los valores posibles de los operandos.

Las operaciones básicas que se pueden realizar sobre datos de este tipo son de dos tipos: lógicas y relacionales. Los operadores lógicos que se pueden aplicar son la negación (!), la conjunción (&&) y la disyunción (||). Para este tipo de operaciones, una forma sencilla de plasmar su resultado al ser aplicados sobre datos de tipos booleano es usar una mesa de verdad.

La tabla 2.1 resume el resultado de aplicar las distintas operaciones lógicas. La negación tiene la particularidad de que sólo se aplica sobre un único dato.

Tabla 2.1. Tabla de verdad de las operaciones booleanas

| operandos |           | Resultados de la operación |           |           |
|-----------|-----------|----------------------------|-----------|-----------|
| A         | B         | A y B                      | A    B    | ! A       |
| FALSO     | FALSO     | FALSO                      | FALSO     | verdadero |
| verdadero | FALSO     | FALSO                      | verdadero | FALSO     |
| FALSO     | verdadero | FALSO                      | verdadero | verdadero |
| verdadero | verdadero | verdadero                  | verdadero | FALSO     |

Si se quiere realizar un resumen de la tabla 2.1, se ve que:

- La negación da como resultado el valor contrario.
- La conjunción es cierta si ambos operandos son ciertos.
- La disyunción es cierta si alguno de los operandos lo es.
- La disyunción exclusiva es cierta sólo si uno de los operandos es cierto, pero no si lo son ambos.

Visto con un ejemplo más concreto, las operaciones lógicas siguen el razonamiento siguiente. Suponga que A y B son interruptores que pueden estar encendidos (true) o no (false). La negación es cómo conmutar a un interruptor. Si estaba encendido pasa a estar apagado y viceversa. La conjunción es cómo preguntar “Es cierto que los dos interruptores están encendidos?”. Sólo hace falta que uno de los dos esté apagado para que la respuesta sea que no (false). La disyunción es cómo preguntar “Es cierto que alguno de los dos interruptores está encendido?”. Si uno de los dos, lo que sea, lo está, la respuesta será si (verdadero).

Respecto a los operadores relacionales, éstos se refieren a las operaciones igual (==) y diferente (!=) , aplicados tal y como indica la tabla 2.2.

Tabla 2.2. Tabla de verdad de las operaciones relacionales entre booleanos

| operandos |           | Resultados de la operación |           |
|-----------|-----------|----------------------------|-----------|
| A         | B         | A == B                     | A != B    |
| FALSO     | FALSO     | verdadero                  | FALSO     |
| verdadero | FALSO     | FALSO                      | verdadero |
| FALSO     | verdadero | FALSO                      | verdadero |
| verdadero | verdadero | verdadero                  | FALSO     |

En este caso, si se vuelve al ejemplo de los interruptores A y B, lo igual es equivalente a preguntar “¿Es cierto que ambos interruptores están en el mismo estado?” (los dos apagados o ambos encendidos). La operación distinta sería el caso contrario: “Es cierto que ¿uno está un apagado y el otro encendido?”.

El siguiente programa muestra, a modo de ejemplo, el resultado de hacer varias operaciones entre datos de tipo booleano. Puede servir para ver más claramente los resultados expresados en las tablas de verdad anteriores.

```
1 clase pública OperacionsBoolea {
2     public static void main (String[] args) {
3         //Operaciones lógicas: el resultado es un booleano
4         System.out.println(!true);
5         System.out.println(true && true);
6         System.out.println(verdadero && falso);
7         System.out.println(true || false);
8         System.out.println(false || false);
9         //Operaciones relacionales: el resultado también es un booleano
10        System.out.println(false == false);
11        System.out.println(false != true);
12    }
13 }
```

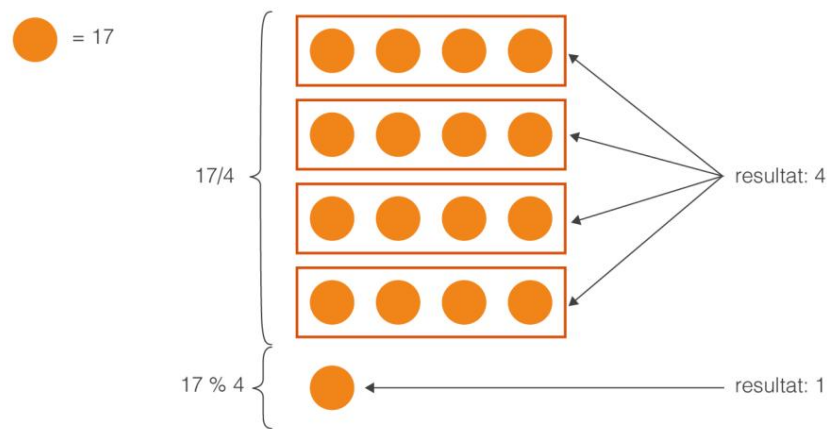
## 2.2.2 Operaciones con números enteros

Las operaciones básicas que se pueden realizar sobre datos de este tipo son de dos tipos: aritméticas y relacionales.

Los operadores aritméticos generalmente soportados son el cambio de signo (-, operador unario), la suma (+), resta (-, operador binario), multiplicación (\*) y división (/).

Cualquier operación entre dos datos de tipo entero siempre da como resultado un nuevo dato también de tipo entero. Ahora bien, dado que los números enteros no disponen de decimales, hay que tener presente que, en el caso de la división, el resultado se trunca y se pierden todos los decimales. En algunos lenguajes de programación también hay una quinta operación llamada módulo (%). El resultado de aplicarla sobre dos enteros devuelve el resto de la operación de división, tal y como esquematiza la figura 2.2.

F igura 2. 2. Esquema del funcionamiento de las operaciones de división y módulo de enteros



La tabla 2.3 muestra algunos ejemplos de resultados de la operación división y módulo en datos de tipo entero.

Tabla 2. 3. Ejemplos de división y módulo entre enteros

| operandos |    | Resultado de las operaciones |       |
|-----------|----|------------------------------|-------|
| A         | B  | A / B                        | A % B |
| 17        | 4  | 4 , en lugar de 4:25         | 1     |
| 9         | 5  | 1 , en lugar de 1.8          | 4     |
| 23        | 10 | 2 , en lugar de 2.3          | 3     |

Los operadores relacionales incluyen el igual (==), diferente (!=), mayor que (>), menor que (<), mayor o igual que (>=) y menor o igual que (<=). En este caso, el resultado de realizar esta operación es un dato de tipo booleano (cierto/falso). La mesa 2.4 muestra algunos ejemplos.

Tabla 2.4. Ejemplos de operaciones relacionales con números enteros

| operandos |    | Resultado de las operaciones |           |           |           |           |
|-----------|----|------------------------------|-----------|-----------|-----------|-----------|
| A         | B  | A == B                       | A > B     | A < B     | A >= B    | A ≤ B     |
| 4         | 3  | FALSO                        | verdadero | FALSO     | verdadero | FALSO     |
| 14        | -2 | FALSO                        | verdadero | FALSO     | verdadero | FALSO     |
| -78       | 34 | FALSO                        | FALSO     | verdadero | FALSO     | verdadero |
| 12        | 12 | verdadero                    | FALSO     | FALSO     | verdadero | verdadero |

El siguiente programa muestra, a modo de ejemplo, el resultado de hacer varias operaciones entre datos de tipo entero.

```
1 clase pública OperationsEnter {
2     public static void main (String[] args) {
3         //Operaciones aritméticas: el resultado es un entero
4         System.out.println(3 + 2);
5         System.out.println(4 - 10);
6         System.out.println(3 * 8);
7         System.out.println(10 / 3);
8         System.out.println(10 % 3);
9         //Operaciones relacionales: el resultado es un booleano
```

```

10     System.out.println(4 == 4);
11     System.out.println(5 > 6);
12     System.out.println(7 < 10);
13 }
14 }

```

### 2.2.3 Operaciones entre reales

Las operaciones que se pueden realizar entre datos de tipo real son exactamente las mismas que entre enteros, tanto aritméticas como relacionales. La única excepción es la operación módulo, que deja de tener sentido, ya que ahora la división sí se realiza con cálculo de decimales. La tabla 2.5 muestra cómo es la división entre datos de tipo real.

Tabla 2. 5. Ejemplos de división entre reales

| operandos |      | Resultado de la operación |
|-----------|------|---------------------------|
| A         | B    | A / B                     |
| 17.0      | 4.0  | 4.25                      |
| 9.0       | 5.0  | 1.8                       |
| 23.0      | 10.0 | 2.3                       |

El siguiente programa muestra, a modo de ejemplo, el resultado de hacer varias operaciones entre datos de tipo real.

```

1 clase pública OperacionesReales {
2     public static void main (String[] args) {
3         //Operaciones aritméticas: el resultado es un entero
4         System.out.println(8.5 + 3.2);
5         System.out.println(5.66 - 3.1);
6         System.out.println(3.1 * 8.4);
7         System.out.println(17.0 / 4.0);
8         //Operaciones relacionales: el resultado es un booleano
9         System.out.println(1.2 == 1.2);
10        System.out.println(-4.3 > -12.45);
11        System.out.println(3.14 < 5.126);
12    }
13 }

```

### 2.2.4 Operaciones entre caracteres

Los datos de este tipo sólo aceptan operaciones relacionales que les permitan compararlos entre ellos, de forma similar a los enteros o reales: igual (==), diferente (!=), mayor que (>), menor que (<), mayor o igual que (>=) y menor o igual que (<=). En los casos de las operaciones que incluyen “mayor que” o “menor que”, se considera su orden en la tabla utilizada para representarlos en el ordenador.

El siguiente programa muestra, a modo de ejemplo, el resultado de hacer varias operaciones entre datos de tipo carácter.

```
1 clase pública OperacionsCaracter { public static void main (String[]  
2     args) {  
3         //Operacions relacionals: el resultat és un booleà System.out.println('a'  
4         == 'a'); System.out.println('a' != 'a');  
5         System.out.println('z' > 'w');  
6         System.out.println('k' < 'b');  
7     }  
8 }  
9 }
```

### 2.2.5 Construcción de expresiones

Como se ha visto, dado un conjunto de datos de un tipo concreto, el mecanismo principal para transformarlos y obtenerlos nuevos es la aplicación de operaciones entre varios operandos. Sin embargo, hasta el momento, los ejemplos de aplicación de operaciones se han limitado a una única operación, con el número de operandos correspondientes según si ésta era unaria o binaria (uno o dos). Ahora bien, en muchos casos, es útil poder aplicar de una sola vez un conjunto de operaciones distintas sobre una serie de datos.

Una expresión es una combinación cualquiera de operadores y operandos.

Para construir una expresión, es necesario que ésta sea correcta en dos niveles, sintáctica y semánticamente, de forma que se respete el significado de los datos usados como operandos y sus operadores. En cualquier caso, las expresiones siempre se escriben en una sola línea, como cualquier texto legible en catalán, ordenando los elementos de izquierda a derecha y de arriba a abajo.

Desde el punto de vista sintáctico, las normas básicas de construcción de expresiones usando literales son las siguientes:

1. Se considera que un literal solo es en sí mismo una expresión.
2. Dada una expresión correcta E, también lo es escribirla entre paréntesis: (Y).
3. Dada una expresión correcta E y un operador unario cualquier op, opE es una expresión correcta.
4. Dadas dos expresiones correctas E1 y E2 y un operador binario cualquiera op, E1 op E2 es una expresión correcta.

Estas normas son un caso general válido para representar cualquier combinación de literales y operaciones, cualquiera que sea su longitud. Ejemplos de expresiones que las siguen, mostradas de forma que se aplican de forma acumulativa, son:

- 6, per a la primera regla.

- $(6 + 5)$ , para la segunda regla, en la que E es  $6 + 5$ .
- $-4$ , para la tercera regla, en la que op es  $-$  y E es  $4$ .
- $6 + 5$ , para la cuarta regla, en la que E1 es  $6$ , E2 es  $5$  y op es  $+$ .
- $(6 + 5) * -4$ , para la cuarta regla, en la que E1 es  $(6 + 5)$ , E2 es  $-4$  y op es  $*$ .

Desde la vertiente semántica, las normas que siempre debe cumplir una expresión son las siguientes:

1. Cualquier operación siempre debe ser entre datos del mismo tipo.
2. La operación usada debe existir para el tipo de dato.

El primer punto es muy importante y hay que tener cuidado al aplicarlo. Por ejemplo, si partimos de literales, escribir un  $6$  corresponde al literal que representa el valor numérico “seis” dentro del tipo de dato entero. El texto  $6.0$  es el mismo pero para los reales, y  $'6'$  (note las comillas simples) para los caracteres. Ahora bien, aunque representan el mismo concepto, no se pueden realizar operaciones entre estos tres literales, ya que pertenecen a tipos diferentes. Sin embargo, note cómo con pequeños detalles de escritura se puede establecer a qué tipo de dato pertenece cada literal dentro de su código fuente directamente. De hecho, todos los ejemplos de valores posibles dado un tipo de dato, mostrados en la sección anterior, son literales.

Las siguientes expresiones son correctas sintácticamente, pero no semánticamente. Por tanto, no se consideran expresiones válidas.

- $5.3 == '4'$  : aunque reales y caracteres disponen de la operación igualdad, ambos literales pertenecen a tipos diferentes (primera regla).
- $true + false$ , la operación suma no existe para los booleanos (segunda regla).
- $'g'$ , la operación cambio de signo no existe para los caracteres (segunda regla).
- $5 == false$ , aunque en booleanos y enteros existe la igualdad, la operación es entre tipos distintos (primera regla).
- $5 || 4.0$ , se realiza una operación entre tipos diferentes y, aparte, la disyunción no existe en los enteros ni en los reales (regla primera y segunda).

### 2.2.6 Evaluación de expresiones

---

Un solo literal entre paréntesis  
es equivalente a  
mismo literal sin  
paréntesis.

---

Entendemos como evaluar una expresión ir aplicando todos sus operadores sobre los distintos datos que la conforman hasta llegar a un resultado final, que es el dato resultante. La evaluación siempre empieza calculando las expresiones que se encuentran entre paréntesis, en orden de más interno a más externo. Una vez que han desaparecido todos los paréntesis, entonces se aplican las operaciones según su orden de precedencia.

El orden de precedencia de un conjunto de operadores es la regla usada por establecer de forma no ambigua el orden cómo resolver las operaciones dentro de una expresión.

Las operaciones con orden de precedencia mayor se evalúan antes que las de orden menor. Para las operaciones descritas hasta ahora, este orden es el siguiente. En caso de empate, se resuelve la expresión ordenadamente de izquierda a derecha. La mesa tabla 2.6 muestra este orden, de mayor prioritario (1) a menos (7).

Tabla 2. 6. Orden de precedencia de los operadores

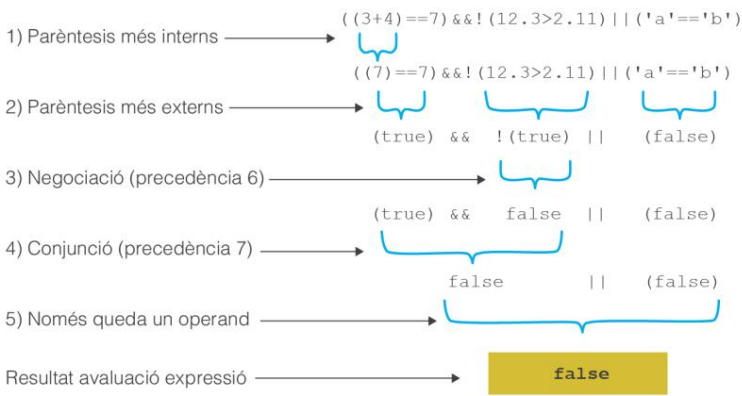
| Orden | Operación                   | Operador  |
|-------|-----------------------------|-----------|
| 1     | Cambio de signo             | -(unari)  |
| 2     | Producto, división y módulo | * / %     |
| 3     | Suma i resta                | + -       |
| 4     | Relacionales de comparación | > < <= >= |
| 5     | Relacionales de igualdad    | == !=     |
| 6     | Negación                    | ! (unari) |
| 7     | Conjunción                  | &&        |
| 8     | Disyunción                  |           |

Aunque existe este orden, es muy recomendable que todas las expresiones basadas en un operador binario siempre se incluyan entre paréntesis cuando deben combinarse con nuevos operadores para generar expresiones complejas, con vistas a mejorar la legibilidad de la expresión general.

En la figura 2.3 se muestra el proceso de evaluación de una expresión de cierta complejidad, que es la que tiene en el siguiente programa:

```
1 //Comprueba una expresión compleja
2 clase pública ProvaExpressio {
3     public static void main(String[] args) {
4         System.out.println(((3 + 4) == 7 )&&!(12.3 > 2.11)||('a' == 'b'));
5     }
6 }
```

F igura 2. 3. Esquema de evaluación de una expresión de ejemplo



Fíjese atentamente cómo en esta expresión aparecen literales de tipos de datos totalmente diferentes. Sin embargo, siempre que se aplica una operación binaria, siempre está entre datos del mismo tipo. De hecho, el resultado de su evaluación es un dato de tipo booleano, pero en lugar de la expresión original había datos booleanos.

Reto 1: realice un programa que evalúe una expresión que contenga literales de los cuatro tipos de datos (booleano, entero, real y carácter) y la muestre por pantalla.

### 2.2.7 Desbordamientos y errores de precisión

Un hecho muy importante que debe tener en cuenta cuando evalúa expresiones aritméticas entre datos de tipo numérico (enteros y reales), es que estos tipos tienen una limitación en los valores que pueden representar. Al haber infinitos números, esto significa que para representarlos todos dentro del ordenador se necesitarían secuencias binarias de infinita longitud, algo imposible. Por tanto, los lenguajes de programación sólo pueden representar un rango concreto en ambos casos. Si alguna vez se supera este rango, se dice que ha sufrido un desbordamiento (overflow) y el resultado de cualquier expresión será siempre incorrecto.

En el caso de los reales, además de existir en número infinito, existe la particularidad de que, dado un número real, éste puede tener infinitos decimales. Esto significa que tampoco es posible representar cualquier secuencia de decimales con número real. Para ciertos valores en realidad se realizan redondeos. La consecuencia directa de este hecho es que al realizar cálculos con datos reales pueden aparecer errores de precisión. Deberá tenerlo en cuenta en programas de cálculo complejos, en los que cada decimal es importante.

Teniendo en cuenta la circunstancia de que el rango de datos que se puede representar usando tipos numéricos está acotado, muchos lenguajes de programación en realidad dividen los tipos enteros y reales en diferentes categorías, cada una definida con una palabra clave diferente. Cada una de estas categorías se considera un nuevo tipo primitivo distinto. El rasgo distintivo para cada caso es el rango de valores que puede alcanzar y la longitud de su representación interna en binario (número de bits).

Para el lenguaje Java, la tabla 2.7 le muestra qué tipos primitivos existen, previendo esta circunstancia. Puede apreciar cuál es el rango aceptable para cada caso y su tamaño. Más allá de estos rangos, ya no es posible operar con tipos primitivos; son necesarias soluciones más complejas.

Los literales que utiliza Java por defecto, y los que se han usado hasta ahora para describir datos de tipo entero y real se corresponden a los tipos entero simple (int) y real de doble precisión (double). Por tanto, a partir de ahora, siempre que se hable de tipos de datos enteros y reales en general, el texto se referirá a estos dos tipos primitivos en concret.

Tabla 2. 7. Rangos y palabras clave de los tipos primitivos numéricos en Java

| Tipo                     | Palabra clave Java | ¿Qué (bit)? | Sonó                                                    |
|--------------------------|--------------------|-------------|---------------------------------------------------------|
| byte                     | byte               | 8           | -128 a 127                                              |
| entrar curt              | corto              | 16          | -32768 a 32767                                          |
| ingresar simple          | entero             | 32          | -2147483648 a 2147483648                                |
| entero largo             | largo              | 64          | -9.223.372.036.854.775.808 a 9.223.372.036.854.775.808  |
| real de simple precisión | flotar             | 32          | -3,40292347*10^38 a 3,40292347*10^38                    |
| real de doble precisión  | doble              | 64          | -4.94065645841246544*10^24 a 1.79769313486231570*10^308 |

Si se quiere indicar que un literal es de otro tipo diferente de estos dos, es necesario indicarlo explícitamente añadiendo al final el carácter 'L', para escribir un long, o 'F', por un float. Java no puede explicitar que un literal es un entero corto. Por ejemplo:

- 5400000000L
- 3.141592F

En Java, un float puede representar fielmente hasta 6 o 7 decimales. Un doble hasta unos 15.

Siempre que se evalúa una operación binaria en la que aparecen literales numéricos de distinto tipo, el tipo del resultado será el mismo que el operando con más sonó.

Por ejemplo, el resultado de evaluar la expresión (2 + 5) \* 40L es un entero de tipo long (concretamente, 280L).

Reto 2: en su entorno de trabajo, cree el siguiente programa. Observe qué ocurre exactamente. Entonces, intente arreglar el problema.

```
1 //Un programa que usa un entero muuuucho grande
2 clase pública TresMilMillones {
3     public static void main (String[] args) {
4         System.out.println(3000000000);
5     }
6 }
```

2.3 Gestión de datos en la memoria

Hasta el momento se ha trabajado con literales en el código fuente de los programas. Ahora bueno, ésta sólo es la punta del iceberg a la hora de operar con datos dentro de un programa. Si se limita a esto, la función del ordenador no va más allá de ser una simple calculadora más complicada de usar. De hecho, los literales sólo son suficientes en los casos que necesita representar datos con un valor conocido en el momento de escribir el programa y que nunca variará a lo largo de la ejecución, para ningún de las ejecuciones.

Ahora bien, en otros muchos casos, el valor de los datos que se quieren tratar dentro de un programa o bien es desconocido (por ejemplo, se trata precisamente del resultado que se está intentando calcular o dependerá de una respuesta del usuario) o bien el valor irá variando a lo largo del programa (por ejemplo, el precio total para pagar en una tienda virtual cuando aún no se ha finalizado la compra). Quizás incluso conozca su valor inicial (en una tienda virtual, el precio cuando no se ha comprado nada todavía seguro que es cero), pero no se puede predecir su evolución. Otro caso que se puede encontrar es que desee guardar los datos resultantes de evaluar una expresión para usarla más adelante, en uno o varios sitios dentro del programa. En ninguno de estos casos se pueden usar literales y lo que se debe usar es una variable almacenada dentro de la memoria del ordenador.

Una variable es un dato almacenado en la memoria que puede ver modificado su valor en cualquier momento durante la ejecución del programa.

### 2.3.1 Cómo funciona la memoria del ordenador

Antes de ver cómo podemos usar variables, vale la pena hacer un repaso de los aspectos más importantes del funcionamiento de la memoria del ordenador desde el punto de vista de cómo la va a usar a la hora de hacer un programa. Esto le ofrecerá una cierta perspectiva sobre algunos de los aspectos más importantes en el uso de las variables.

Suponga que está en la escuela primaria y que un buen día el profesor le saca a la pizarra y le dice que tendrá que resolver una suma de 4 números enteros de 8 cifras. Ante todo, lo que haréis es buscar un pedazo de pizarra que esté libre para poder escribir vosotros, que no implique tener que borrar aquellas cosas que el profesor ha escrito antes. Concretamente, necesita espacio suficiente para poder escribir los 4 números que le dictará el profesor y para poder escribir la solución final. Si no se viera capaces de calcular todo de memoria, también podría buscar espacios para poder realizar cálculos auxiliares.

Después, a medida que el profesor te va dictando los números, usted los va escribiendo en la pizarra para no olvidarlos. Una vez los tiene todos apuntados, ya puede proceder a realizar las operaciones pertinentes y ver si se sale o no. Mientras realiza los cálculos, puede ir escribiendo, si le va bien, nuevos números que representan cálculos parciales, siempre que tenga espacio en la pizarra. También puede borrar o modificar en todo momento la información que ha escrito en la pizarra. Por último, con la ayuda de todos estos datos, obtenga el resultado final. Una vez dice el resultado final, y el profesor le dice si es correcto o no, borre la parte de la pizarra donde ha escrito para dejar espacio a sus compañeros y vuelva a su sitio.

Conceptualmente, la memoria del ordenador tal y como la utilizaría un programa no es muy diferente de vosotros y de esta pizarra. Es un espacio disponible en el que los diferentes programas pueden guardar libremente datos para poder llevar a cabo su tarea. Ahora bien, los dos puntos más importantes que este símil tiene en común con la memoria de un ordenador, y que debe tener en cuenta, son:

- La memoria es un espacio compartido entre todos los programas que están en ejecución, como el sistema operativo. Por tanto, antes de poder almacenar ningún dato, es necesario poder buscar un espacio vacío que no se esté usando.
- Los datos almacenados no son persistentes, sólo existen mientras el programa está resolviendo la tarea en curso. Al terminar, se borran. En el caso de un programa, sus datos sólo están presentes en la memoria mientras éste está en ejecución. Cuando termina, los datos desaparecen totalmente. La única manera de evitarlo es que, antes de terminar la ejecución, sean trasladadas a un medio persistente (por ejemplo, en el caso de la pizarra, en una hoja de papel, y en el caso del ordenador, en un archivo).

Ahora bien, por la naturaleza de sistema digital de un ordenador, la forma en que su memoria organiza todos los datos contenidos no es exactamente como una pizarra.

También hay dos aspectos en los que el símil diverge y que hay que tener presentes:

- Recuerde que los datos se representan en formato binario cuando se encuentran dentro del ordenador. Un ordenador sólo puede representar internamente la información en ceros y unos, de acuerdo con el estado de los transistores de sus chips.
- De hecho, los datos se organizan en celdas dentro de la memoria, de forma más parecida a una hoja de cálculo. Cada una puede contener 8 bits de información. Ahora bien, un dato puede ocupar más de una celda y el número exacto que ocupa depende de su tipo y del lenguaje usado.

---

Cada celda individual está indexada con un identificador numérico llamado dirección.

---

### 2.3.2 Declaración de variables

De la descripción del funcionamiento de la memoria, el punto más importante es que para poder disponer de una variable dentro de su programa, primero se debe buscar un espacio libre en la memoria, compuesto de varias celdas, que será asignado a esta variable de forma exclusiva. Afortunadamente, los lenguajes de programación de nivel alto ofrecen un sistema relativamente sencillo para ello.

---

El lenguaje de programación ya se encarga internamente de todos los aspectos de nivel bajo vinculados a la estructura real de la memoria.

---

Toda variable dentro del código fuente de un programa debe haber sido declarada previamente por el programador antes de poder utilizarla.

Para declarar una variable dentro de su programa, sólo es necesario especificar tres cosas: el tipo, un identificador o etiqueta único y un valor inicial. Aunque la sintaxis para declarar una variable puede variar según el lenguaje de programación, casi siempre es necesario definir estas tres cosas.

El identificador nos permite diferenciar las diferentes variables que haya dentro del código fuente del programa, el tipo delimita qué valores puede almacenar y qué operaciones son aplicables, y el valor inicial el contenido que habrá en cuanto se ha declarado. Evidentemente, el valor inicial debe ser un dato del

mismo tipo que la variable. Una vez completada correctamente la declaración, inmediatamente después de esa línea de código, es posible usar desde entonces en adelante el identificador escogido para leer el valor de los datos almacenados o sobrescribirlos.

#### La importancia de inicializar las variables

No todos los lenguajes requieren asignar un valor inicial a una variable. En estos casos, simplemente no se puede predecir cuál es el valor almacenado en la memoria (la secuencia de ceros y unos en ese espacio). Puede ser cualquier valor, por lo que el resultado de cualquier operación en la que intervenga esta variable será impredecible. Por este motivo, aunque el compilador acepte la sintaxis como correcta si no se realiza, es un hábito muy bueno inicializar siempre las variables.

En caso de duda, siempre puede asignar el 0 como valor inicial.

La sintaxis para declarar una variable en una línea de código fuente Java es la siguiente.

Recuerde que las palabras clave de Java para los cuatro tipos primitivos presentados son boolean, int, double y char:

```
1 paraulaClauTipus identificadorVariable = valorInicial;
```

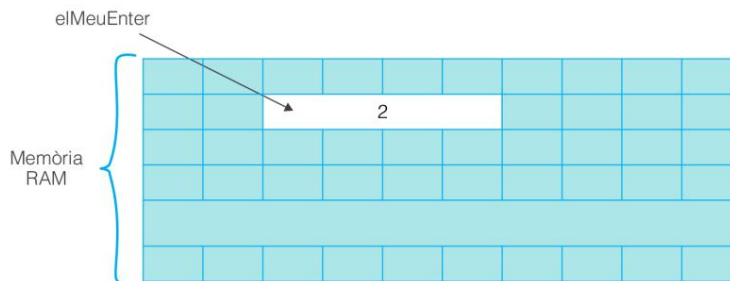
Estrictamente, se considera que la parte a la izquierda del signo igual es la declaración propiamente de la variable, mientras que la parte derecha es la inicialización: la especificación de cuál es el valor inicial.

La forma más sencilla de verlo es mediante un ejemplo concreto:

```
1 //El programa que sólo declara una variable de tipo entero 2 public class DeclararEnter { public static void  
main(String[] args) { int MiEnter = 2; //Ahora existe una  
3     variable en memoria que contiene el valor entero 2.  
4  
5  
6     }  
7 }
```

Nada más ejecutar esta instrucción, la memoria quedaría como muestra la figura 2.4. En este caso, esta variable ocupa dos celdas, puesto que Java almacena los enteros en la memoria usando 32 bits. En cualquier caso, esto es totalmente transparente para el programador.

Figura 2.4. Memoria de la computadora al declarar la variable "elMeuEnter"



También es posible declarar más de una variable del mismo tipo de dato en una sola línea separando los identificadores e inicializaciones con comas. Por ejemplo:

```
1 //El programa que declara de cop dues variables de tipus enter
2 clase pública DeclararEnter {
3     public static void main(String[] args) {
4         int elMeuEnter = 2, elMeuAltreEnter = 4;
5         //Ahora hay dos variables en la memoria
6         //Contienen los valores enteros 2 y 4, respectivamente.
7     }
8 }
```

### 2.3.3 Identificadores

Un identificador toma la forma de un texto arbitrario. Puede elegir el que más le guste. De todas formas, es muy recomendable que siempre use algún texto que os ayude a entender fácilmente qué se está almacenando en cada variable, ya que esto ayuda a entender qué hace el código fuente de su programa. Cuando un programa es largo o hace mucho que no se ha editado, incluso puede ser problemático para su propio creador volver a entender para qué servía cada variable.

---

Coloquialmente, por referir-se a un identificador también se suele usar el término nombre. Ejemplo: "El nombre de una variable".

---

Nótese las diferencias entre los dos códigos siguientes. Aunque el código de declaración de variables es totalmente equivalente (se declaran tres variables de tipo real), seguro que en uno resulta más fácil saber qué hace el programa y qué papel tiene cada variable en el proceso sin ni siquiera tener que conocer el resto de las instrucciones.

```
1 //Programa A. Un código poco comprensible.
2 clase pública DivideixlSuma {
3     public static void main(String[] args) {
4         doble x = 20.0;
5         doble y = 6.0;
6         doble z = 3.0;
7         //Ahora vendría el resto del código
8         ...
9     }
10 }
```

```
1 //Programa B. Un código más comprensible.
2 clase pública DivideixlSuma {
3     public static void main(String[] args) {
4         dividendo doble = 20.0;
5         divisor doble = 6.0;
6         doble sumaAlFinal = 3.0;
7         //Ahora vendría el resto del código
8         ...
9     }
10 }
```

No tenga pereza de usar identificadores de variables comprensibles, aunque sean algo más largos. Evidentemente, la clave de todo ello siempre está en el equilibrio. Usar un identificador que ocupe cien letras y no se pueda leer en una sola línea de texto tampoco ayuda mucho.

Desafortunadamente, no cualquier identificador se considera válido. Cada lenguaje de programación impone unas condiciones sobre el formato que considera admisible. Aunque aquí se explica el caso concreto del lenguaje Java, no hay muchas variaciones entre los distintos lenguajes. Un identificador:

- No puede contener espacios.
- No puede empezar con un número.
- Es desaconsella usar accents.
- No puede ser igual que alguna de las palabras clave del lenguaje.

Una palabra clave (o reservada) de un lenguaje de programación es aquella que tiene un significado especial dentro de su sintaxis y se usa para componer ciertas partes o instrucciones en el código fuente de un programa.

El conjunto de palabras reservadas en Java se enumera a continuación:

abstracto, continuar, para, nuevo, cambiar, afirmar, predeterminado, ir a, paquete, sincronizado, booleano, hacer, si, privado, esto, romper, doble, implementa, protegido, lanzar, byte, si no, importar, público, lanza, caso, enumeración, instancia de, devolver, transitorio, atrapar, extender, entero, corto, intentar, carácter, final, interfaz, estático, vacío, clase, finalmente, largo, fp estricto, volátil, constante, flotante, nativo, super, mientras

Fíjese que en esta lista hay algunas palabras que ya ha visto, usadas para declarar variables o especificar dónde comienza el código fuente o el método principal del programa: boolean, int, double, char, public, class, void, etc.

#### Convenciones de nomenclatura

Aunque tiene libertad total para elegir los identificadores de las variables, al igual que ocurre con los nombres de las clases de Java, existe una convención de código para nombrarlos en Java.

En este caso es necesario usar lowerCamelCase (notación de camello minúscula). Esta notación es como UpperCamelCase, pero en ese caso la primera palabra siempre va en minúscula y no en mayúscula.

Ejemplos de identificadores de variables que siguen la convención son: divisor, resultadoDivisio, MiEnter, etc.

Por último, recuerde que Java es sensible a mayúsculas y minúsculas, por la que cosa MiEnter y miEnter se consideran identificadores totalmente diferentes.

### 2.3.4 Ús de variables

La principal valía de una variable como pieza fundamental dentro de un programa es su capacidad para almacenar datos y recuperar su valor en cualquier momento a lo largo del código fuente del programa, una vez declaradas. La recuperación de los datos almacenados es el mecanismo más directo.

Cuando una variable ha sido declarada, su identificador puede usarse exactamente igual que un literal dentro de cualquier expresión. El valor que representará será el del valor almacenado en memoria en ese instante para aquella variable.

---

En el caso de Java, el delimitador de fin de bloque es la clave `}`.

---

Ahora bien, es necesario tener presente que en la mayoría de lenguajes de programación la declaración de una variable sólo tiene validez desde la línea de código en la que se declara hasta encontrar el delimitador que marca el final del bloque de código en el que se ha declarado. Fuera de ese rango de instrucciones del código fuente, su ámbito, es como si no estuviera declarada.

El ámbito de una variable es el contexto bajo el que se considera declarada.

Si se intenta hacer uso de un identificador que no corresponde a ninguna variable declarada (ya sea porque nunca se ha declarado o porque se utiliza fuera de su ámbito), al compilar el código fuente se producirá un error. Hay que decir que el uso de variables fuera de su ámbito no es un problema que ahora mismo tenga que preocuparle mucho, ya que los programas que se están tratando se componen de un único bloque que engloba todas las instrucciones (el método principal).

El siguiente programa divide dos datos de tipo real y después suma una tercera usando variables en lugar de literales:

```
1 //Un programa que calcula una división y una suma, completado 2 public class DivideYSuma { public static void
main(String[] args) {
3
4     double dividendo = 20.0; double
5     divisor = 6.0; double
6     sumarAlFinal = 3.0; //Las variables se
7     pueden usar como literales en una expresión.
8     //Aquí es equivalent a fer (20.0/6.0) + 3.0
9     System.out.println((dividendo/divisor) + sumarAlFinal);
10 }
11 }
```

En este ejemplo, el uso de las variables es correcto, ya que se accede a su identificador en una línea de código que se encuentra entre la línea donde se han declarado y la clave de final de bloque del método principal. El uso se ha realizado en su ámbito.

Ahora bien, el uso de variables no tiene mucho sentido cuando para realizar una operación se podrían usar literales directamente y el resultado sería el mismo, como en el ejemplo anterior. La verdadera utilidad consiste en poder cambiar el valor almacenado en memoria. El valor de una variable puede modificarse en cualquier momento dentro de un programa con el operador de asignación (=):

```
1 identificadorVariable = expresión;
```

Esta sintaxis ya la había visto, ya que es prácticamente igual a la usada para establecer el valor inicial al declarar la variable. De hecho, una variable también puede inicializarse a partir de una expresión, no es necesario usar un único literal.

---

Si el tipo del resultado de la expresión y el de la variable en la que se asigna no es el mismo, habrá un error de compilación.

---

Al hacer una asignación a una variable, lo primero que sucede es que se evalúa

la expresión que hay en el lado derecho. El resultado obtenido entonces se convierte de inmediato en el nuevo valor almacenado dentro de la variable. El valor anterior se pierde para siempre, puesto que queda sobrescrito. Ahora bien, recuerde que el operador de asignación (=), como cualquier otro operador binario, siempre requiere que sus operandos pertenezcan al mismo tipo de dato. Por tanto, sólo es correcto asignar expresiones que evalúen un tipo idéntico al utilizado al declarar la variable.

Alerta. No es lo mismo el operador de asignación (=) que el de igualdad (==). Es necesario no confundirlos.

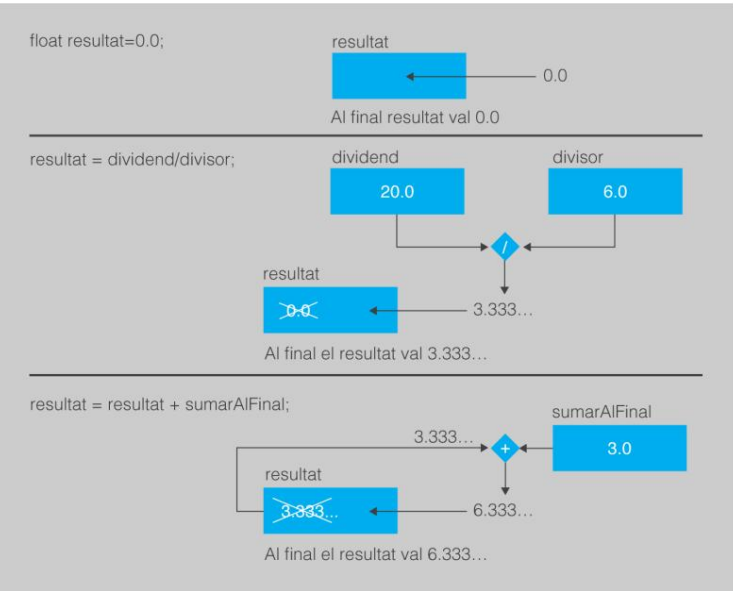
Es muy importante que sea consciente del orden para resolver una asignación. Primero es necesario evaluar la expresión y después realmente modificar el valor de la variable, ya que dentro de una expresión se puede usar el identificador de la misma variable en la que se está haciendo la asignación! Por tanto, en el lado derecho se usa su valor original, y una vez hecha la asignación, ésta se habrá modificado de acuerdo con el resultado de evaluar la expresión.

La mejor manera de ver el comportamiento de una variable cuando se modifica el valor es con un ejemplo:

```
1 //Un programa que calcula una división y una suma, pero poco a poco
2 clase pública DivideixlSumaDetallat {
3     public static void main(String[] args) {
4         dividendo doble = 20,0;
5         divisor doble = 6.0;
6         doble sumaAlFinal = 3.0;
7         double resultado = 0.0;
8         //El resultado será una variable que se irá modificando.
9         resultado = dividendo/divisor;
10        //¡Una expresión que usa la misma variable que se modifica!
11        resultado = resultado + sumarAlFinal;
12        System.out.println(resultado);
13    }
14 }
```

La figura 2.5 muestra un esquema de cómo se va modificando la variable resultado en cada paso.

Figura 2.5. Cambio del valor de la variable “resultado”



Para el caso de la conjunción y disyunción, los operadores de ese tipo son &= y |=.

### Operación y asignación simultánea

Modificar el valor de una variable a partir de aplicar una operación sobre su valor es algo que ocurre muy a menudo dentro de un programa. Por este motivo, algunos lenguajes (como Java) ofrecen una serie de atajos llamados operadores de operación, aritmética o lógica, y de asignación. En éste se combina una única operación directamente con la asignación de acuerdo con el siguiente formato:

Donat un operador binari op,  $x \text{ op} = y$  equival a  $x = x \text{ op} y$ .

Por tanto, en el ejemplo anterior también se podría calcular el resultado final haciendo:

```
resultado += sumarAlFinal;
```

Reto 3: haga un programa con dos variables que, sin usar ningún literal en ninguna parte excepto para inicializar estas variables, vaya calculando e imprimiendo sucesivamente los 5 primeros valores de la tabla de multiplicar del 4. Puede usar operadores aritméticos y de asignación, si lo desea.

### 2.3.5 Constantes

Al definir variables, existe la posibilidad de asignar un identificador que le permite dar una cierta semántica al valor que almacena. Si elige bien este identificador, puede ser mucho más sencillo entender qué hace el programa al leer el código fuente.

Desgraciadamente, al usar literales se pierde un poco esta expresividad, ya que sólo se dispone del valor tal cual, pero no se puede saber exactamente su función sin inspeccionar el código con atención. Para solucionar este detalle, los lenguajes de programación permiten reemplazar el uso de literales por constantes.

---

Un código bien comentado también ayuda a entender el código fuente de un programa.

---

Una constante es un tipo especial de variable que tiene la particularidad de que dentro del código del programa su valor sólo puede ser leído pero nunca modificado.

Hay varias maneras de definir constantes en el Java, pero verá la más popular.

En este caso, las constantes se definen fuera del método principal, pero dentro del blog que identifica un código fuente Java. La sintaxis es idéntica a la definición de la variable, pero antes de la palabra clave para el tipo de dato es necesario añadir las palabras reservadas `private static final` en este orden. O sea:

---

El uso de este término es similar al que se hace en matemáticas o física, donde se usa para indicar ciertos valores universales e inmutables:  $\pi$  (3,1415...),  $c$  (300.000 m/s, la velocidad de la luz), etc.

---

```
1 private static final palabraClaveTipus identificadorVariable = valorInicial;
```

Al igual que con las variables, el valor inicial puede ser tanto literal como expresión. Sin embargo, esta expresión no puede contener variables. De todas formas, lo normal es usar literales, por lo que nos referiremos a ello a lo largo de las explicaciones.

### Convenciones de nomenclatura

Por el caso de las constantes, las convenciones de código existentes en Java son diferentes. En éste siempre se usan mayúsculas para todas las letras y, en caso de nombrarlas con una composición de diferentes palabras, éstas se separan con un símbolo de subrayado (underscore, `_`).

Ejemplos de identificadores de constantes que siguen la convención son: `CONSTANTE`,

UNA\_CONSTANTE, MI\_CONSTANTE, etc.

Compare la legibilidad del código de los dos programas siguientes. En el primero parece que se aplique una multiplicación arbitraria al final de código, pero en el segundo queda claro por qué hacerla.

```
1 //Calcula el precio de comprar varias cosas
2 clase pública CalculaPreu {
3     public static void main(String[] args) {
4         doble preuFinal = 0.0;
5         //Va sumando precios de productos
6         ...
7         preuFinal = preuFinal + (preuFinal * 0.18);
8         System.out.println(preuFinal);
9     }
10 }

1 //Calcula el precio de comprar varias cosas, y queda más claro cómo lo hace
2 clase pública CalculaPreu {
3     privado estático final doble IVA = 0.18;
4     public static void main(String[] args) {
5         doble preuFinal = 0.0;
6         //Va sumando precios de productos
7         ...
8         preuFinal = preuFinal + (preuFinal * IVA);
9         System.out.println(preuFinal);
10    }
11 }
```

Las constantes son también especialmente útiles cuando, aparte de querer etiquetar ciertos literales de modo que el código sea más fácil de entender, el literal en cuestión aparece varias veces en el código fuente. Suponga que en una versión posterior del programa es necesario realizar un cambio en el valor de este literal. Por ejemplo, el IVA necesario aplicar dentro de un programa de tienda virtual pasa de 0,18% a 0,16%. Si no se ha declarado como constante, habrá que buscar este valor línea por línea y asegurarse de que se ha usado para realizar un cálculo de IVA (quizás se ha usado este mismo literal, pero con otra finalidad). Si lo ha declarado, basta con modificar sólo la declaración de la constante. El cambio se produce automáticamente en todas las expresiones en qué se usa la constante con ese identificador.

La declaración de constantes en lugar de usar directamente un literal cuando éste aparece en muchos sitios dentro del código también ofrece otra ventaja sutil. Suponga que le equivoca al escribir alguna de las apariciones del literal. En lugar de 0,18% de IVA escriba 0,28% (ya que las teclas '1' y '2' están juntas). En este caso, el programa compilará perfectamente, pero no hará lo que debe hacer correctamente. Lo peor es que, de entrada, no tendrá ni la menor idea de por qué ocurre esto y tendrá que repasar todo el código hasta encontrar el literal incorrecto. En cambio, con una constante, el literal lo escriba una única vez y ya está. En cambio, si alguna vez se equivoca al escribir el identificador de la constante, el compilador considerará que está haciendo uso de un identificador no declarado previamente y le dará un error, e indicará exactamente el sitio dentro del código fuente donde se ha equivocado. Arreglarlo será muy fácil.

La mejor forma de verlo es con un ejemplo, que para hacerlo más interesante incorpora el uso de variables. A continuación hay dos versiones distintas del código fuente un programa que hace exactamente lo mismo. Si le dicen que en ambos hay una error, ¿en cuál de los dos casos le sería más sencillo identificarlo?

```
1 //¿Qué debe hacer este programa? Compila perfectamente, ¡pero tiene un error!  
2 class pública Conversión {  
3     public static void main(String[] args) {  
4         valor doble = 12.0;  
5         System.out.println(valor*1.3656);  
6         valor = 300.0;  
7         System.out.println(valor*1.3756);  
8         valor = 189.0;  
9         System.out.println(valor*1.3656);  
10    }  
11 }
```

```
1 //Quizás ya está más claro qué hace el programa. Para encontrar el error, sólo es necesario  
   compilar-lo.  
2 class pública ConversioError {  
3     public static final double CONVERSIÓN_EURO_A_DOLAR = 1.3656;  
4     public static void main(String[] args) {  
5         valor doble = 12.0;  
6         System.out.println(valor*CONVERSIÓN_EURO_A_DÓLAR);  
7         valor = 300.0;  
8         System.out.println(valor*CONEVRSIO_EURO_A_DOLAR);  
9         valor = 189.0;  
10        System.out.println(valor*CONVERSIÓN_EURO_A_DÓLAR);  
11    }  
12 }
```

Intente compilar los dos programas. De hecho, para encontrar el error en el segundo ni tan sólo es necesaria una inspección detallada del código por su parte, ya que aparecerá un error de compilación y le dirá exactamente dónde se encuentra. Ahora suponga también que el valor de conversión ha cambiado y el euro vale 1,4 dólares. ¿En cuál de los dos programas ¿es más sencillo hacer el cambio? Pues imaginaos lo mismo en un programa con cientos o miles de líneas de código.

Una pregunta que se puede hacer es si no se puede lograr exactamente lo mismo simplemente usando una variable, y no modificar nunca su valor dentro del programa. Es una buena pregunta, y la respuesta es que a efectos funcionales sería lo mismo. Ahora bien, definir constantes tiene dos ventajas, pero para entenderlo es necesario tener algo de idea de cómo funciona un compilador. La primera ventaja es que el compilador controla explícitamente que ninguna constante declarada es modificada en alguna instrucción del código fuente programa. Si esto sucede, considera que el código fuente es erróneo y genera un error de compilación. Esto puede ser muy útil para detectar errores y garantizar que sus constantes realmente nunca ven modificado su valor a lo largo de la ejecución del programa a causa de alguna distracción. El segundo es que, en algunos lenguajes, antes de procesar el código fuente, el compilador reemplaza todas las apariciones del identificador de cada constante por el literal que se le ha asignado como valor. Por tanto, al contrario que las variables, las constantes no ocupan espacio realmente en la memoria mientras el programa se ejecuta.

Por tanto, es una buena costumbre ceñirse al uso de constantes cuando se está tratando con datos generales que nunca cambian de valor.

Reto 4: haga dos programas, uno que muestre por pantalla la tabla de multiplicar del 3, y otro, la del 5. Los dos deben ser exactamente iguales, letra por letra, excepto en un único literal dentro de todo el código.

## 2.4 Conversiones de tipo

Hasta el momento se ha dicho que siempre que existe una asignación de un valor, ya sea al establecer el valor de una variable o el de una constante, los tipos debe ser exactamente igual en ambos lados de la asignación. Bien, en realidad esto no es estrictamente cierto. Bajo ciertas condiciones es posible realizar asignaciones entre tipos de datos diferentes si sobre el valor en cuestión se puede realizar una conversión de tipos.

Este proceso también se conoce como casting.

Una conversión de tipos es la transformación de un tipo de dato en otro de diferente.

Dentro de los lenguajes de programación existen dos tipos diferentes de conversiones de tipos: las implícitas y las explícitas. El primer caso corresponde a aquellas conversiones fáciles de resolver y que el lenguaje de programación es capaz de gestionar automáticamente sin problemas. El segundo caso es más complejo y lo fuerza el programador, si cabe.

### 2.4.1 Conversión implícita

Este caso es el más simple y se fundamenta en que existen ciertas compatibilidades entre tipos primitivos de datos diferentes. La mejor manera de ilustrarlo es con el siguiente programa:

```
1 public class ConversioTipus { public static void main (String[]  
2     args) { //El literal "100" es un entero y "real" es un float. Son tipos diferentes,  
3  
4         ¿no?  
5         float real = 100; //Pero,  
6         "doble" y "real" son dos variables de tipo diferente, ¿no? double doble = real; System.out.println(doble);  
7  
8     }  
9 }
```

La sintaxis de este programa no se corresponde con lo que se ha explicado hasta ahora. En cambio, si intenta compilarlo y ejecutarlo, funcionará correctamente. Esto se debe a que en este código existen los dos casos de conversiones implícitas más habituales.

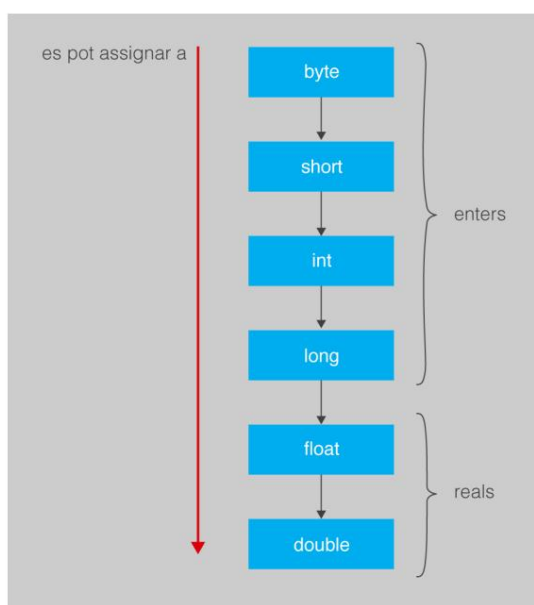
El primer caso se corresponde con que todos los valores que pertenecen al tipo de dato entero se pueden traducir muy fácilmente a real. Simplemente es necesario considerar que sus decimales son .0 y ya está. Esto es lo que ocurre aquí. El literal 100 es un entero (concretamente un int), pero se asigna sobre una variable de tipo float, por lo que en principio sería incorrecto. Habría que usar el literal que expresa ese valor como un real de precisión simple, el 100.0F. Sin embargo, el compilador de Java considera que la asignación es correcta, ya que es capaz de realizar automáticamente esta conversión desde el tipo entero a real.

El segundo caso está vinculado al rango del tipo de dato numérico. Si un valor es de un

tipo primitivo numérico concreto, éste puede ser interpretado fácilmente como un dato de los tipos que acepta un mayor rango de valores. Por ejemplo, cualquier valor de tipo `int` que pueda imaginar seguro que también está dentro de los valores aceptados entre los `long`. Igual ocurre entre los `float` y los `double`. El compilador de Java también se da cuenta de ello y por este motivo acepta una asignación de un valor `float` (menos rango) a un `double` (más rango) sin problemas, ya que sabe realizar la conversión.

La figura 2.6 hace un resumen de las relaciones entre tipos numéricos. Las relaciones son transitivas, por lo que todo dato de un tipo numérico puede ser asignado a variables o constantes de cualquiera de los tipos posteriores en la relación.

Figura 2.6. Relaciones de compatibilidad entre tipos numéricos



En cualquier caso, a lo largo del texto siempre se usarán los literales del tipo correcto al realizar asignaciones.

#### 2.4.2 Conversión explícita

En cualquier caso, donde haya asignaciones entre datos de tipo diferente pero el lenguaje elegido sea incapaz de realizar una conversión implícita, el compilador siempre dará error. Ya de por sí, la aparición de este error suele significar que algo no está bien y que es necesario modificar el programa. Pero hay casos en los que es permisible eliminar este error especificando en el código que se quiere forzar la conversión de los datos a un nuevo tipo de todas formas. Básicamente, lo que hace es obligar al compilador a transformar un dato de un tipo en uno diferente “lo mejor posible”. Para indicárselo, ante la expresión para convertir se pone el tipo de dato resultante que se quiere obtener entre paréntesis. La sintaxis es la siguiente:

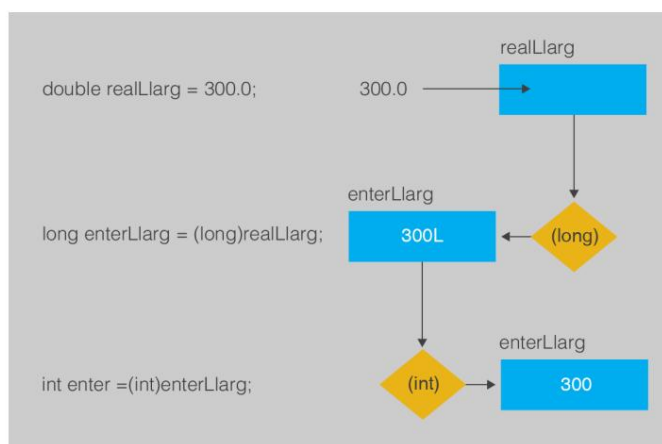
<sup>1</sup> identificador = (palabraClauTipus)expresión;

Al hacer esto, el resultado de la expresión se intentará convertir al tipo de dato especificado, sea cual sea el tipo en el que evalúe originalmente. El valor final obtenido dependerá mucho de la situación bajo la que se realiza la conversión, ya que es evidente que convertir datos a tipos totalmente distintos de su origen no es una acción directa. En cualquier caso, el tipo elegido para la conversión será compatible con el de la variable en la que se asigna, como siempre.

La conversión explícita debe usarse con mucho cuidado y teniendo muy claro qué se está haciendo.

La situación más frecuente en la que se usa la conversión explícita es para invertir el orden establecido en las relaciones de la figura 2.7 (que muestra un esquema de lo que está sucediendo en cada línea del código), ya que normalmente un lenguaje de programación no permite realizar asignaciones en dirección opuesta a las flechas.

Figura 2. 7. Conversión explícita entre variables



Esto se debe a que, por ejemplo, un real o un valor de tipo long muy grande no pueden traducirse normalmente como un entero (int). En el primer caso no hay forma de representar los decimales y en el segundo se sale fuera del rango admisible.

Ahora bien, sí existen casos en los que esta asignación tiene sentido: cuando los decimales de lo real son exactamente .0 o cuando el valor del long está dentro del rango aceptable para los enteros. Entonces, usando la conversión explícita puede evitar que el compilador dé un error y hacer que intente convertir de forma correcta los valores en enteros. Si puede garantizar que el valor original cumple las condiciones para que la traducción sea correcta, la conversión se completará con éxito. En caso contrario, el valor final asignado será erróneo.

A continuación se muestra un ejemplo de conversiones explícitas entre tipos que se supone que no son compatibles. Ahora bien, puesto que el programador puede garantizar que el valor real para convertir, 300.0, es representable en diferentes tipos de entero, todo funciona correctamente. Al final se ve por pantalla el valor 300 (o sea, un literal de tipo entero). Pero si borra las conversiones explícitas, (long) y (int), el programa no compilará.

```
1 public class ConversioExplicita { public static void main (String[] args)
2     { double realLargo = 300.0; //Asignación incorrecta. Un real tiene decimales, ¿no?
3
4
```

```
5    long enterLargo = (long)realLargo; //Asignación
6    incorrecta. Un entero largo tiene un rango mayor que un entero, ¿no? int enter = (int)enterLargo;
7    System.out.println(enter);
8
9    }
10 }
```

Para que este mecanismo funcione es muy importante que el programador esté seguro de que el valor para convertirlo realmente pertenece al tipo de dato de destino.

De lo contrario, el resultado final no se adaptará fielmente al dato original. Por ejemplo, en el caso de la conversión de cualquier tipo real en entero se perderán todos los decimales (se truncará el número), y en el caso de asignar un valor fuera del rango posible entre cualquier tipo de dato obtendrá un desbordamiento.

Reto 5: experimenta qué pasa si en el programa anterior se inicializa la variable real Largo con un valor con varios decimales. ¿El programa sigue compilando? ¿Qué resultado da? Luego pruébelo asignando un valor superior al rango de los enteros (por ejemplo, 3000000000.0).

### 2.4.3 Conversión con tipos no numéricos

En ciertos lenguajes de programación también se puede intentar realizar conversiones de tipos en las que alguno o ambos datos son de tipo no numérico. Entonces lo que se pide es realizar una operación a bajo nivel con la representación interna, en binario, del dato. Para entender qué sucede en esta situación, cabe recordar que la combinación de ceros y unos cómo se representa un dato concreto depende de su tipo. Ahora bien, también debe tenerse presente que una misma secuencia binaria puede representar valores diferentes para tipos de datos diferentes. Por ejemplo, el valor de tipo carácter 'a' se codifica internamente con la secuencia binaria 0000000001100001, pero esta secuencia también es usada para codificar el valor de tipo entero '97'. El tipo de dato proporciona un contexto sobre el significado de los datos en binario al leer un dato dentro del ordenador.

#### El lenguaje del ordenador y el lenguaje natural

Para ejemplificar qué significa que una misma representación de un dato la pueden compartir muchos contextos diferentes, se puede hacer de nuevo un símil con el lenguaje natural. Supongamos que encuentra un papel en el que está escrita la palabra clave. En este símil, el papel representa la memoria, y clave, el dato almacenado. ¿Sin otra información pueden ser capaces de saber su significado? En realidad, con estas cuatro letras uno puede referirse a muchas cosas: una llave para abrir una puerta, un clavo para colgar un cuadro, la llave para descubrir un misterio (una pista), una llave de artes marciales, etc. De hecho, la palabra clave tiene más de quince significados. El tipo de dato correspondería al contexto bajo el que debe interpretarse esta palabra.

Hacer una conversión con datos de tipo no numérico indica al ordenador que debe tomar el valor binario encontrado en la memoria tal cual e interpretarlo como otro tipo de dato totalmente diferente.

En la mayoría de casos, éstas deben realizarse explícitamente, pero en situaciones

muy concretas el lenguaje es incluso capaz de interpretarlas implícitamente. A modo de ejemplo, Java es capaz de llevar a cabo la conversión de carácter a enter, y viceversa, de forma implícita. En otros lenguajes debe hacerse explícitamente o simplemente no se puede hacer.

```
1 public class ConversioCaracter { public static void main (String[] args)
2
3     { int enter = 'a'; char character = 98; //Escribe el valor binario del literal 'a' interpretado
4       como un entero: 97.
5
6       System.out.println(enter); //Escribe
7       el valor binario del literal 98 interpretado como un carácter: 'b
8
9       System.out.println(personaje);
10    }
```

En otros lenguajes, como C, se puede intentar con prácticamente cualquier tipo dado.

Cabe decir que el uso de este caso es muy marginal. De hecho, a pesar de esta última opción de operar a bajo nivel, en cada lenguaje existen conversiones explícitas que simplemente son imposibles. Si las intenta hacer, el compilador dirá que existe un error igualmente. En Java por ejemplo, es el caso de convertir un dato del tipo booleano a cualquier otro.

## 2.5 Visualización y entrada de los datos en Java

Lo cierto es que no tiene ningún sentido que un programa lleve a cabo el procesamiento de unos datos si esta tarea después no tiene ninguna repercusión en el sistema de entrada/salida. Sobre todo teniendo en cuenta que una vez finalizado el programa, todos los valores tratados se borran de la memoria. Lo mínimo que puede hacer es mostrar el resultado de la tarea por pantalla, por lo que el usuario que la ha puesto en marcha sabe cuál ha sido el resultado de todo. Tampoco tiene sentido usar variables si en éstas siempre se almacenan expresiones fruto de operaciones entre literales. A efectos prácticos, podrían usarse ya directamente literales siempre. Otras acciones más complejas ya serían guardar los datos en un archivo de forma persistente, imprimirlos o que éstos sirvan como señal de control sobre algún periférico externo.

Hay muchas maneras de entrar o mostrar los datos al usuario, basta con ver las interfaces gráficas de los sistemas modernos, pero aquí verá la más simple: cadenas de texto en pantalla o por teclado.



Todo dato mostrado por un periférico está representado por datos dentro del ordenador.

### 2.5.1 Instrucciones de salida de datos por pantalla

Aunque hasta el momento ya se han encontrado en los ejemplos de apartados anteriores algunas instrucciones que sirven para mostrar cosas por pantalla, ha llegado el momento de inspeccionarlas con algo más de detenimiento. Antes de empezar, sin embargo, vale la pena decir que cada lenguaje de programación tiene sus propias instrucciones para visualizar datos por pantalla. Si bien la idea general puede tener ciertas

similitudes, la sintaxis puede ser totalmente distinta. Por tanto, hay que tener en cuenta que las descripciones se centran exclusivamente en cómo funciona el lenguaje Java.

Principalmente, Java proporciona dos instrucciones para mostrar datos en la pantalla:

```
1 //Muestra en pantalla el resultado de evaluar la expresión //x//.
2 System.out.print(x); 3 //Hace lo mismo,
pero al final añade un salto de línea.
4 System.out.println(x);
```

Cuando escriba con el teclado, el texto se muestra en el mismo orden como vaya llamando a las instrucciones de salida por pantalla. No se puede solicitar un posicionamiento absoluto en cualquier sitio de la pantalla. Por tanto, cada instrucción empezará a escribir inmediatamente después de donde la ha dejado la instrucción anterior.

Uno de los aspectos más importantes de representar datos por pantalla es que este periférico sólo trata con texto. O sea, conjuntos de caracteres en Java (char). Por tanto, en el proceso de salida a la pantalla todo dato que se quiere mostrar debe sufrir una transformación desde la representación original hasta el conjunto de caracteres que la representan en formato texto.

Introduzca un texto

Suponga que tenemos una variable de tipo entero en Java (int), en la que está almacenado el valor 1309. Esto internamente se representa con el valor binario: 0000010100010100. Ahora bien, para la pantalla lo que se tendrá que mostrar son los 4 caracteres (char) '1', '3', '0', '9'. Es decir, debe ordenarse al sistema de entrada/salida por pantalla que muestre los cuatro valores de la representación en binario:

- '1': 00000000000000000000000000000001
- '3': 00000000000000000000000000000011
- '0': 00000000000000000000000000000000
- '9': 00000000000000000000000000000001

Por tanto, si se quiere escribir un entero, haría falta un paso de transformación previo a representarlo en diferentes caracteres individuales. En lo que se refiere a este tipo de instrucciones, una de las grandes ventajas de Java, como lenguaje de nivel alto en relación con otros lenguajes de nivel más bajo, es que las dos instrucciones básicas de que dispone pueden aceptar cualquier expresión que al ser evaluada resulte en cualquier tipo primitivo, sea cual sea (entero, real, booleano o carácter).

Automáticamente se encargan ellas mismas de transformar el valor almacén-mate en su representación en formato texto, y la muestran por pantalla.

Por ejemplo, haciendo lo siguiente:

```
1 //Muestra diferentes expresiones simples por pantalla. 2 public class MostrarExpresiones { public
static void main(String[] args) { //Muestra el texto "3" por pantalla.
3
4
5     System.out.println(3); //
6     Mostra el texto "a" por pantalla.
7     System.out.println("a"); //Mostra
8     el texto "3.1416" por pantalla.
```

---

Otros lenguajes ofrecen  
otros mecanismos, algunos  
no tan simples, para  
realizar esta transformación.

---

```
9      double x = 3.1416;  
10     System.out.println(x); //Muestra  
11     el texto "true" por pantalla. boolean a = true; boolean  
12     b = false;  
13     System.out.println((a||  
14     b)&&(true));  
15     }  
16 }
```

Se muestra por pantalla lo siguiente:

```
1 3  
2 a  
3 3.1416  
4 verdadero
```

### 2.5.2 Tipo de dato avanzados: cadenas de texto

En muchos casos, seguramente no bastará con sólo escribir la representación textual de un tipo de dato simple. Les gustaría escribir una frase entera del estilo “El resultado obtenido es...”. Es decir, escribir blogs de texto. Partiendo de esta necesidad, si reflexiona un poco, puede que llegue a la conclusión de que el tipo de dato carácter no es muy útil, ya que para mostrar frases largas, con lo que ha visto hasta el momento, habría que hacerlo carácter por carácter, de manera muy molesta.

En realidad, el tipo de dato carácter no suele usarse directamente, sino que los lenguajes de programación lo usan como base para generar un nuevo tipo de dato más avanzado que sirva para procesar texto.

La palabra clave  
para identificar este tipo  
de dato en Java es String.

Una cadena de texto (String) es un tipo de dato que permite representar cualquier secuencia de caracteres de longitud arbitraria.

- Ejemplos de valores de una cadena de texto: “¡Hola, mundo!”, “El resultado final es...”, etc.
- Ejemplos de datos que suelen representarse con una cadena de texto: cualquier mensaje de texto por pantalla.

Para referirnos a un dato de tipo cadena de texto, lo escribimos entre comillas dobles ("). Tenga cuidado, ya que no es lo mismo 'a' que "a". La primera es un literal que representa un dato de tipo carácter y la segunda es de tipo cadena de texto.

Las cadenas de texto no son un tipo primitivo.

Cada lenguaje ofrece su forma de gestionar cadenas de texto y representarlas por pantalla. Desde el ejemplo del programa “¡Hola, mundo!”, ya habéis visto que las

instrucciones de salida básica de Java por pantalla también pueden mostrar cadenas de texto de Java sin problemas, exactamente al igual que se hace con los tipos primitivos.

```
1 System.out.println("¡Hola, hermano!");
```

Por último, recuerde que, como tipo de dato, también se pueden declarar variables usando la palabra clave, al igual que con cualquier otro tipo primitivo:

```
1 //Muestra el texto "¡Hola, mundo!" por pantalla.
2 clase pública MostrarHolaMon {
3     public static void main(String[] args) {
4         String holaMon = "Hola, món!";
5         System.out.println(holaMon);
6     }
7 }
```

## Operadores de las cadenas de texto

En contraposición de los tipos primitivos, no se puede generalizar al hablar de operaciones de cadenas de texto. Cada lenguaje puede ofrecer diferentes (o ninguna). En el caso del Java, las cadenas de texto aceptan el operador suma (+), por lo que se pueden generar expresiones de la misma forma que se puede hacer con los tipos primitivos. En este caso, el resultado de aplicarlo en una operación entre cadenas de texto es que éstas se concatan. Se genera una nueva cadena de texto formada por la unión de cada uno de los operandos consecutivamente.

```
1 //Muestra el texto "¡Hola, mundo!" por pantalla usando concatenación de cadenas de
  texto.
2 clase pública HolaMonConcatenat {
3     public static void main(String[] args) {
4         String hola = "Hola,";
5         Cadena mon = "món";
6         Cadena exclamacion = "!";
7         //Muestra el texto "¡Hola, mundo!" por pantalla
8         System.out.println(hola + mon + exclamacion);
9     }
10 }
```

Nótese que las cadenas de texto se pegan directamente una detrás de otra, sin añadir ningún espacio.

De hecho, para facilitar aún más el trabajo del programador, Java va algo más allá y permite aplicar este operador entre una cadena de texto y cualquier tipo primitivo. El resultado siempre es una nueva cadena de texto en la que el tipo primitivo se transforma en su representación textual.

```
1 //Muestra el resultado de una división simple entre reales.
2 public class Dividir{
3     public static void main(String[] args) {
4         dividendo doble = 18954,74;
5         divisor doble = 549,12;
6         double resultatDivisio = dividendo/divisor;
7         String mensaje = "El resultado obtenido es " + resultatDivisio + ".";
8         System.out.println(mensaje);
9     }
10 }
```

Las dos últimas líneas también pueden convertirse en una sola si se usa directamente la expresión sobre la instrucción de salida:

```
1 System.out.println("El resultado obtenido es " + resultatDivisio + ".");
```

Personajes de control

En algunos lenguajes de programación, como en Java, hay un conjunto de literales dentro de los caracteres que son especialmente útiles dentro de las cadenas de texto, ya que permiten representar símbolos o acciones con un significado especial. Estos se llaman secuencias de escape y siempre se componen de una contrabarra (\) y un carácter adicional. La tabla 2.8 muestra una lista de los más habituales.

| Tabla 2. 8. Secuencias de escape típicas | Secuencia de escape | Acción o símbolo representado         |
|------------------------------------------|---------------------|---------------------------------------|
|                                          | \t                  | Una tabulación.                       |
|                                          | \n                  | Un salto de línea y regreso de carro. |
|                                          | \'                  | El carácter "cometa simple".          |
|                                          | \"                  | El carácter "comillas dobles" (").    |
|                                          | \\                  | El carácter "contrabarra" (\).        |

Como se puede apreciar, para los dos primeros casos es necesario usar una secuencia especial, ya que son acciones que encontramos en un teclado, pero no son gráficamente representables. Por ejemplo, el código:

```
1 //Muestra texto usando caracteres de control.  
2 clase pública CaractersControl{  
3     public static void main(String[] args) {  
4         System.out.println("Línea 1 iLínea 2 iLínea 3");  
5     }  
6 }
```

mostraría por pantalla:

```
1 Línea 1  
2     Línea 2  
3 Línea 3
```

En el resto de casos, su existencia se debe a la forma en que se representan los literales de caracteres o cadenas de texto. Debido a que el literal de un carácter se rodea con dos comillas simples y el de una cadena de texto con comillas dobles, es la única modo de distinguir dentro del texto si este símbolo se usa para delimitar un literal o como texto que forma parte. La contrabarra se representa de esta forma para distinguirla respecto de cuándo se usa, precisamente, para especificar una secuencia de escape. Así pues, el código:

```
1 //Muestra una cadena de texto que incluye comillas dobles.  
2 clase pública HolaMonCometes{  
3     public static void main(String[] args) {  
4         System.out.println("¡Hola \"món\"!");  
5     }  
6 }
```

mostraría por pantalla:

```
1 Hola "mundo"!
```

Repte 6: feu un programa que mostri en pantalla de manera tabulada la taula de verdad de una expresión de disyunción entre dos variables booleanas.

### 2.5.3 Entrada simple de datos por teclado

Explicar la entrada de datos por teclado cuando apenas se ha empezado a aprender a programar usando Java tiene una serie de complicaciones, ya que para entender exactamente la sintaxis de cada instrucción usada es necesario conocer numerosos conceptos que todavía no se han contado. Además, para acabar de complicarlo, también interviene el aspecto de lenguaje orientado a objetos de Java. Por este motivo, de momento basta con que simplemente aprenda las instrucciones desde un punto de vista puramente práctico y funcional, pero sin tener que saber exactamente qué está pasando o qué sintaxis concreta se está usando. Siempre que sea necesario, puede usar como plantilla el código fuente de ejemplo en este apartado.

Para leer datos de teclado es necesario usar una biblioteca, un conjunto de extensiones en las instrucciones disponibles por defecto en el lenguaje. Por usar alguna de las extensiones disponibles en una biblioteca, cada una identificada con un nombre, primero es necesario importarla. Entonces, lo primero que se hace es inicializar la extensión, asignándole un identificador, ya partir de entonces ya la puede usar dentro de su codi font.

Escriba y ejecute el siguiente ejemplo para ver cómo puede entrar por teclado dos números enteros y realizar una operación entre ellos.

```
1 //Esta línea hace que la biblioteca esté disponible.
2 import java.util.Scanner;
3 //Un programa que lee un entero y lo muestra por pantalla.
4 clase pública Sumar {
5     public static void main (String[] args) {
6         //S'inicialitza la biblioteca.
7         Scanner lector = new Scanner(System.in);
8         //Se pone un mensaje de bienvenida.
9         System.out.println("Anem a sumar dos nombres enters");
10        //Se lee un valor entero por teclado.
11        //Se espera que se pulse la tecla de retorno.
12        System.out.print("Escribe un número y pulsa la tecla de retorno: ");
13        int primerEnter = lector.nextInt();
14        lector.nextLine();
15        //Se vuelve a hacer...
16        System.out.print("Torna a fer-ho: ");
17        int segundoEnter = lector.nextInt();
18        lector.nextLine();
19        //Hacemos la operación.
20        entero resultado = primeraEntrada + segundaEntrada;
21        //Imprime el resultado por pantalla!
22        //S'usa l'operador suma entre una cadena de text i un enter
23        System.out.println("La suma de ambos valores es " + resultado + ".");
24    }
25 }
```

Recuerde que el código fuente debe estar en un archivo llamado Sumar.java.

Una vez que la extensión está inicializada (en este caso, sólo se hace una sola vez), el programa está listo para leer datos de diferentes tipos desde el teclado. De

En realidad, lector es un identificador intercambiable con cualquier otro.

momento, basta que aprenda cómo puede leer datos de una en una, usando diferentes líneas de texto para entrar cada dato individual. Para ello, es necesario alternar las instrucciones de lectura enumeradas en la tabla 2.9, según el tipo de dato que se desea leer desde el teclado, y la instrucción `lector.nextLine()`; tal y como se ve en el ejemplo.

Tabla 2. 9. Instrucciones para la lectura de datos por teclado

| Instrucción                       | Tipo de dato leído |
|-----------------------------------|--------------------|
| <code>lector.nextByte()</code>    | byte               |
| <code>lector.nextShort()</code>   | corto              |
| <code>lector.nextInt()</code>     | entero             |
| <code>lector.nextLong()</code>    | largo              |
| <code>lector.nextFloat()</code>   | flotar             |
| <code>lector.nextDouble()</code>  | doble              |
| <code>lector.nextBoolean()</code> | booleano           |
| <code>lector.next()</code>        | Cadena             |

Para no tener que entrar en detalles, puede considerar que cada una de las instrucciones de la tabla se comporta exactamente igual que una expresión que como resultado evalúa el dato introducido por el teclado. La propia instrucción es la que se encarga de transformar el texto escrito, una cadena de texto, en un dato del tipo correspondiente. Normalmente, todo dato leído será asignado a alguna variable previamente declarada. Si le resulta más cómodo, puede hacerse en una sola línea como en el código de ejemplo, asignando el valor leído directamente como parte de la inicialización de la variable.

Usando este procedimiento, si en una misma línea se introduce texto con más de un dato, sólo se lee el primero de ellos. El resto se ignora.

Este mecanismo no permite leer caracteres individuales. Recuerde, sin embargo, que desde el punto de vista del tipo de dato, una cadena de texto que contiene un único carácter no es lo mismo que un carácter individual. Otro aspecto importante es que la forma en que reconoce los valores reales, si interpreta los decimales separados con un punto o una coma (por ejemplo, 9.5 o 9,5), depende de la configuración local del sistema. En las configuraciones catalana y española se separan con una coma.

Si hace pruebas se dará cuenta de que en este programa, si se equivoca e introduce un valor diferente de un entero, el programa se detiene inmediatamente y muestra un mensaje de error por pantalla. Esto es normal, ya que el dato escrito por teclado debe encajar con el tipo de dato que lee la instrucción usada. También puede comprobar que, al igual que al evaluar expresiones, intentar asignar un dato que excede el rango admisible para el tipo de una variable resultará en un desbordamiento.

En el caso de entrada por el teclado, todos estos hechos son especialmente complicados de controlar, ya que nunca sabe qué escribirá realmente el usuario, expresamente o por equivocación. Por el momento, puede considerar que todo esto no es ningún problema. Por ahora, simplemente, procure no entrar nunca datos incorrectos por el teclado.

**Reto 7:** realice un programa que muestre por pantalla la multiplicación de tres números reales entrados por teclado.

---

Este mensaje indica que ha sucedido un error de ejecución (error de runtime).

---

## 2.6 Soluciones de los retos propuestos

### Reto 1:

```
1 public class EvaluarExpresionesLiterals {
2     public static void main (String[] args) {
3         //booleà == ((enter > (enter + enter))&&(caràcter == caràcter))|(real <
            real)
4         System.out.println(false == ((6 > (5 + 3))&&('a' == 'b'))|(4.4 < 10.54) );
5     }
6 }
```

### Reto 2:

```
1 clase pública TresMilMillones {
2     public static void main (String[] args) {
3         //Hay que decir que el literal se corresponde a un "long" (añadir una "l" al final)
4         System.out.println(3000000000l);
5     }
6 }
```

### Reto 3:

```
1 clase pública IniciTaulaMultiplicarQuatre {
2     public static void main (String[] args) {
3         entero a = 4;
4         entero b = 4;
5         System.out.println(a);
6         a = a + b;
7         System.out.println(a);
8         a = a + b;
9         System.out.println(a);
10        a = a + b;
11        System.out.println(a);
12        a = a + b;
13        System.out.println(a);
14    }
15 }
```

### Reto 4:

```
1 public class TablaTreslCinco {
2     //Para hacer la del 5, cambiar el 3 por un 5.
3     public static final int TAULA = 3;
4     public static void main (String[] args) {
5         System.out.println(1*TAULA);
6         System.out.println(2*TAULA);
7         System.out.println(3*TAULA);
8         System.out.println(4*TAULA);
9         System.out.println(5*TAULA);
10        System.out.println(6*TAULA);
11        System.out.println(7*TAULA);
12        System.out.println(8*TAULA);
13        System.out.println(9*TAULA);
14        System.out.println(10*TAULA);
15    }
16 }
```

## Reto 5:

```
1 clase pública ConversioExplicitaDiversosDecimals {
2     public static void main (String[] args) {
3         doble realLlarg = 3000000000.0;
4         //Asignación incorrecta. Un real tiene decimales, ¿no?
5         long enterLlarg = (long)realLlarg;
6         //Asignación incorrecta. Un entero largo tiene un rango mayor que un entero,
7         //¿No?
8         int enter = (int)enterLlarg;
9         System.out.println(enter);
10    }
```

## Reto 6:

```
1 clase pública TaulaVeritatDisjuncio {
2     public static void main(String[] args) {
3         System.out.println("A      \tB \t(A || B)");
4         System.out.println("false\tfalse\tfalse");
5         System.out.println("verdadero\tfalse\tverdadero");
6         System.out.println("false\ttrue \ttrue");
7         System.out.println("verdadero \tverdadero \tverdadero");
8     }
9 }
```

## Reto 7:

```
1 import java.util.Scanner;
2 clase pública MultiplicarEntradaTeclat {
3     public static void main(String[] args) {
4         //S'inicialitza la biblioteca.
5         Scanner lector = new Scanner(System.in);
6         //Se pone un mensaje de bienvenida.
7         System.out.println("Vamos a multiplicar tres números reales");
8         //Se lee un valor entero por teclado.
9         //Se espera que se pulse la tecla de retorno.
10        System.out.print("Escribe un número y pulsa la tecla de retorno: ");
11        double primerReal = lector.nextDouble();
12        lector.nextLine();
13        //Se vuelve a hacer...
14        System.out.print("Torna a fer-ho: ");
15        double segonReal = lector.nextDouble();
16        lector.nextLine();
17        //Se vuelve a hacer...
18        System.out.print("I un altre cop: ");
19        double tercerReal = lector.nextDouble();
20        lector.nextLine();
21        //Hacemos la operación.
22        double resultat = primerReal * segonReal * tercerReal;
23        //Imprime el resultado por pantalla!
24        System.out.println("La multiplicación de los valores es " + resultado + ".");
25    }
26 }
```